# Advanced Computing Architectures

## 5 – Memory organization design

- Memory hierarchy description: the locality principles
- Registers
- On-chip cache and RAM memories
- Improving cache performances: multi-level caches
- The “main” DRAM memory
- Hints on storage memories

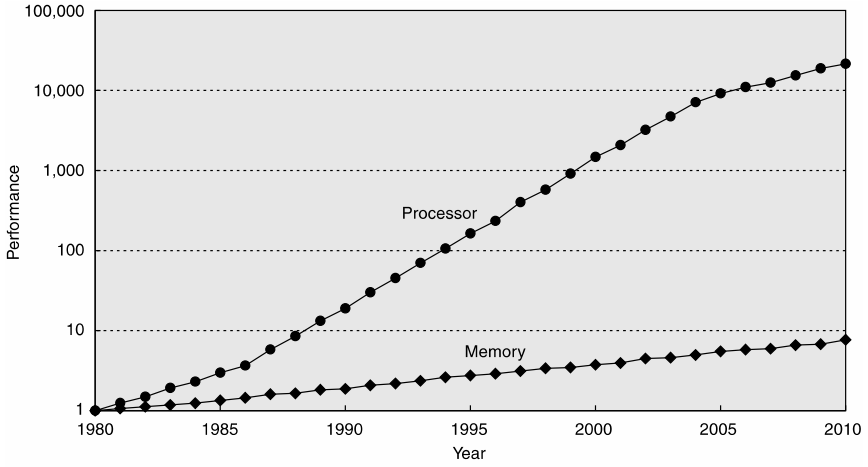
# General requirements and problems

## Objectives

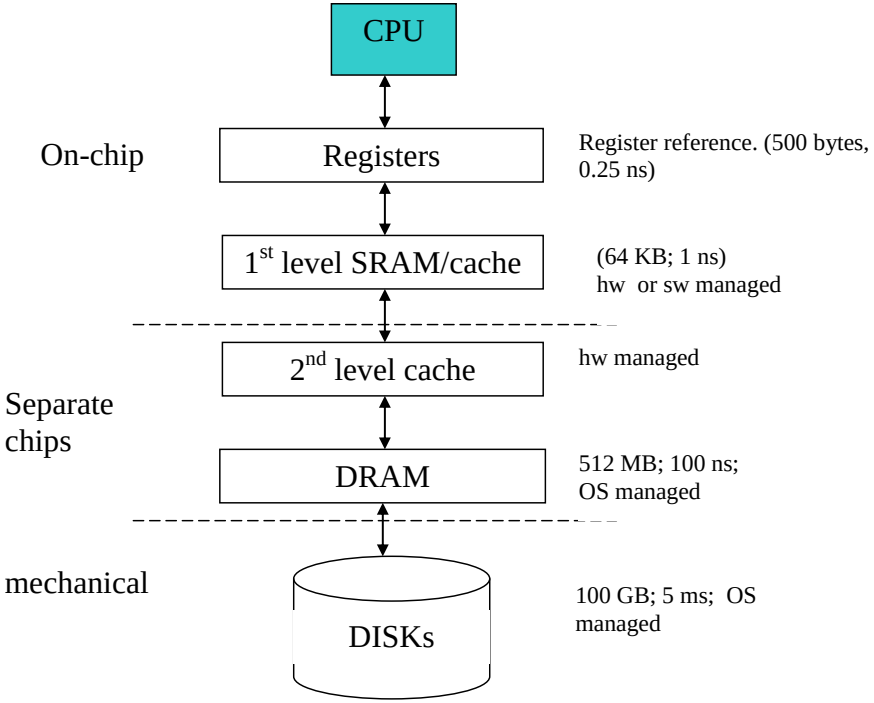
- Applications require ***ever larger memories*** as well as ***ever higher performance***
- Having a very large and fast memory (propagation time of memory similar to the speed of the computational logic) at a ***reasonably low price***

## Problems

- Larger memories are slower than smaller memories
- Fast memories are much more expensive than slow memories
- Increasing gap between memory and CPU performance (see picture)
  - 1980-1986: DRAM latency decreases by 7%/year, CPU performance increase 135 %/year
  - 1986-2000: CPU performance has improved by 1,55 times/year



# Solution: memory hierarchy




Instead of using a single large memory, modern systems use a ***hierarchy***.

- Characterized by different technology, cost, dimensions and access mechanisms
- **The programmer sees a unique large memory space**
- **The CPU sees a fast memory at acceptable cost!**

At the top level, memories are much smaller than the typical application requires.
At run- time, ***transfers between different levels*** become necessary.

Hierarchies work well because memory access follows the principle of ***locality***

# Locality - 1

**Two kinds of locality**

- ***Temporal locality***: when a memory entry is referenced, with high probability it will be referenced again within a short time.
  - **Example:** repeated access to instructions and variables within a loop
- ***Spatial locality***: whenever a memory entry is referenced, with high probability reference will be made shortly to neighboring items
  - **Example:** instructions within a sequence; data organized as arrays, etc…

- Memory hierarchy design is based on both locality principles and Amdahl's law: the most probably used data and instructions have to be stored so that they can be accessed as quickly as possible

# Locality - 2

- Locality of instructions is generally true, regardless of the kind of application, in which the computing system is used. In fact, as a rule of thumb, a program spends 90% of its execution time in only 10% of the code
- Instruction locality is maximum in tight loops and minimum in case of frequent procedure calls
- Data locality instead is much more application-dependent. In some applications, *data* exhibit locality of *one kind only*

***Examples***

– *"Streaming" data in image (video) processing*: there is no temporal locality for such data, as they are used once (or just a few times) and then deleted

– Filter coefficients used in signal or image processing: the *same* coefficients are used repeatedly in time

Special locality requirements and application-specific needs may lead to some modifications in memory hierarchy design

# Different types of memory hierarchies

Memory hierarchy features largely depend on the class of computing systems considered.

- **Desktop computers** are primarily running one or more applications on top of an operating system for a single user → desktop computers are concerned mainly with average latency from the memory hierarchy. Of course, they require mass storage (disks) at the lowest layers of the hierarchy to store application data.
- **Server computers** may typically have hundreds of users potentially running many applications simultaneously. These characteristics result in more context switches → not only the average latency of data is important but also memory bandwidth and data protection → larger memory and redundant mass storage devices, to prevent data loss
- **Embedded computers** require different memory design principles:
  1. Often real-time applications and programmers must worry about worst-case latencies.
  2. Embedded applications are concerned about power and battery life. Hence, embedded computers usually do not employ hardware-intensive optimizations
  3. Embedded applications typically run only one application and use a very simple operating system (if any). Hence, the protection role of the memory hierarchy is often diminished.
  4. The main memory itself is often very small, disk storage is unnecessary.

# Memory hierarchy: summary

Latency of first access

| Level | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Name | registers | cache | main memory | disk storage |
| Typical size | < 1 KB | < 16 MB | < 512 GB | > 1 TB |
| Implementation technology | custom memory with multiple ports, CMOS | on-chip or off-chip CMOS SRAM | CMOS DRAM | magnetic disk |
| Access time (ns) | 0.25–0.5 | 0.5–25 | 50–250 | 5,000,000 |
| Bandwidth (MB/sec) | 50,000–500,000 | 5000–20,000 | 2500–10,000 | 50–500 |
| Managed by | compiler | hardware | operating system | operating system/ operator |
| Backed by | cache | main memory | disk | CD or tape |

Time to read full block

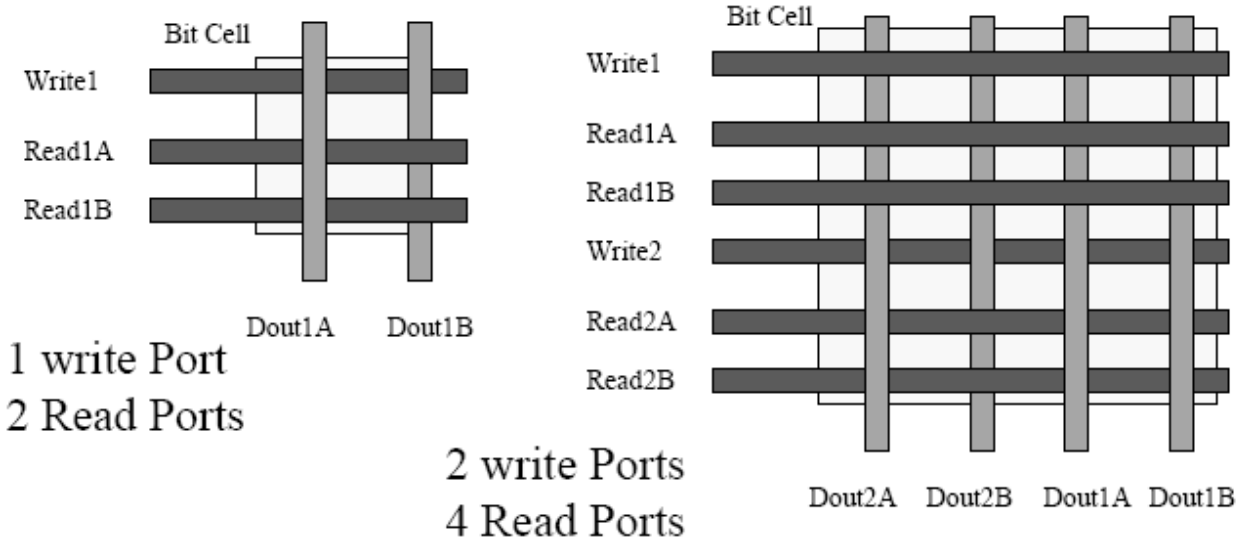
# The register file

- Registers are usually "**exposed**" to the programmer. Alternatively, the compiler allocates variables to registers.
- Registers are usually placed into a **regular *register file***. A register file consists of multiple read/write ports to allow parallel, simultaneous data accesses
- Typically, at least **2 read and 1 write ports** are required to access operands of a single instruction
- By **increasing the number of functional units** within a processor, the **number of ports** of the register file as well as **the number of registers has to grow** to avoid performance bottleneck
- For instance, if we have *N* ALUs, the register file should have 2*N read* ports and *N write* ports to perform simultaneous operations
- Unfortunately, the size of the register file grows with the square of the number of ports → an excessive number of ports slows down the processors

# Multi-port advantages and limitations

This quadratic increase in chip area is mostly due to the new additional input and output lines:

- Routing problems
- Internal bit cell loading increases → larger power consumption
- Linear increment in delay with the number of ports




As a rule of thumb, it is not profitable to have a register file with more than 15-20 ports: 12 is actually the best tradeoff when number of FU is larger than 4

# On-chip cache and RAM memories

# Main features

- The **first-level** (usually on-chip) memory is the fastest kind of memory after registers where most probably data and instructions should be stored.
- It is based on **Static RAM** (SRAM) and its size is very limited
- Two solutions: usual **SRAM** and **cache** memories.

## Usual SRAM

- It is "**exposed**" to programmer and compiler
- Data transfers from/to memory are performed by **software**
- It is mostly used in low-end or specialized embedded processors where **worst-case latency** to recover data and power are the main issues

## Cache

- Usually it is "**transparent**" to programmer and compiler
- Transfers from/to lower-level memories are performed by **hardware**
- It is mostly used in high-end processors where the power is less critical and **average performance** should be maximized

**Von Neumann scheme**

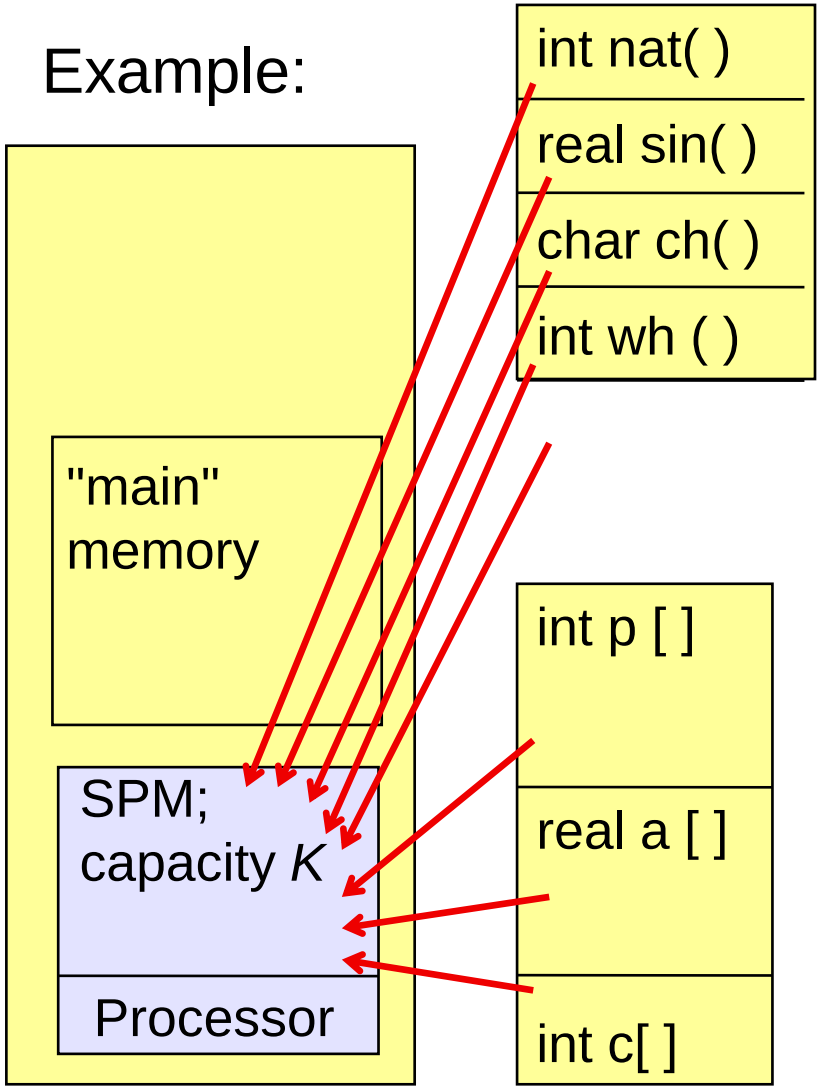
**Harvard scheme and its variants**

**Unified cache**

**Split cache**

# A low power solution: Scratch Pad

Example:




- **The scratch pad** is a **buffer** of memory (no cache) where most frequently used program functions instructions and data are allocated
- This approach requires a preliminary "profiling" to decide what instructions have to be placed into the scratch pad
- Allocation usually performed statically at compile time (no HW support)
- This kind of solution is very used in embedded systems because the program is usually fixed and it is a reasonable tradeoff between performance and power consumption

# Cache basics

## *Purpose*

- A cache is a special kind of memory based on SRAM cells which is used to **store the data having the maximum probability to be used**
- When the CPU asks for data from memory, it looks for it in the cache first. Cache control logic determines whether the required item is present or not. If not, suitable transfers from lower-level memories must be effected
- Lower-level memories can be either the main DRAM memory or a further level of cache. Level 2 (L2) cache is larger and slower than L1 (can be either on-chip or off-chip) but it can greatly improve global performance

## *Structure*

- Cache can be either ***unified*** (i.e., simultaneously instruction- and data-cache) or ***split*** into I-cache and D-cache

# Some definitions

- ***Hit***: the item requested by the CPU is present in cache
- ***Miss***: the item requested by the CPU is ***not*** present in cache
- ***Hit rate***: **fraction** of memory accesses rewarded by hit
- ***Miss rate***: **fraction** of memory accesses resulting in a miss
  (*miss rate = 1 - hit rate*)
- ***Hit time***: access time (or clock cycles) to cache in the case of success (includes time to determine whether access is met by hit or miss)
- ***Miss penalty***: time (or clock cycles) required to substitute a block of data in cache with another block from the lower-level storage
- ***Miss time***: = ***miss penalty + hit time***, time required to obtain requested data in the case of miss

# Cache performance

- Cache performance is given in terms of average memory access time (AMAT)

$$
\begin{aligned}
AMAT &= hit_rate * hit_time + miss_rate * miss_time \\
&= hit_rate * hit_time + miss_rate * (hit_time + miss_penalty) \\
&= hit_time + miss_rate * miss_penalty
\end{aligned}
$$

  - ***Miss penalty*** is largely determined by technology
  - ***Miss rate*** instead can be reduced by means of architectural decision
  - The main goal in cache design is to keep the hit rate as large as possible (so, miss rate as low as possible) so that AMAT converges towards the hit time

# Cache performance

- Cache performance affects CPU time

$$CPU_{time} = \left(CPU_{execution_cycles} + MEM_Stalls_{cycles}\right) \cdot T_{CLK}$$

$$= IC \cdot \left(CPI_{execution} + \frac{MEM_Stalls_{cycles}}{IC}\right) \cdot T_{CLK}$$

$$= IC \cdot \left(CPI_{execution} + \frac{MEM_{accesses} \cdot miss_rate \cdot miss_penalty}{IC}\right) \cdot T_{CLK}$$

$$= IC \cdot \left(CPI_{execution} + \frac{MEM_{accesses}}{IC} \cdot miss_rate \cdot miss_penalty\right) \cdot T_{CLK}$$

**Total CPI**

# Cache performance

- Consider a CPU with CPI = 1.0. Load and stores are 50% of the instructions. Miss penalty is 25 clock cycles, and miss rate is 2%
- What is the total CPI?

$$CPU_{time} = \left(CPU_{execution_cycles} + MEM_Stalls_{cycles}\right) \cdot T_{CLK}$$

$$= IC \cdot \left(CPI_{execution} + \frac{MEM_{accesses}}{IC} \cdot miss_rate \cdot miss_penalty\right) \cdot T_{CLK}$$

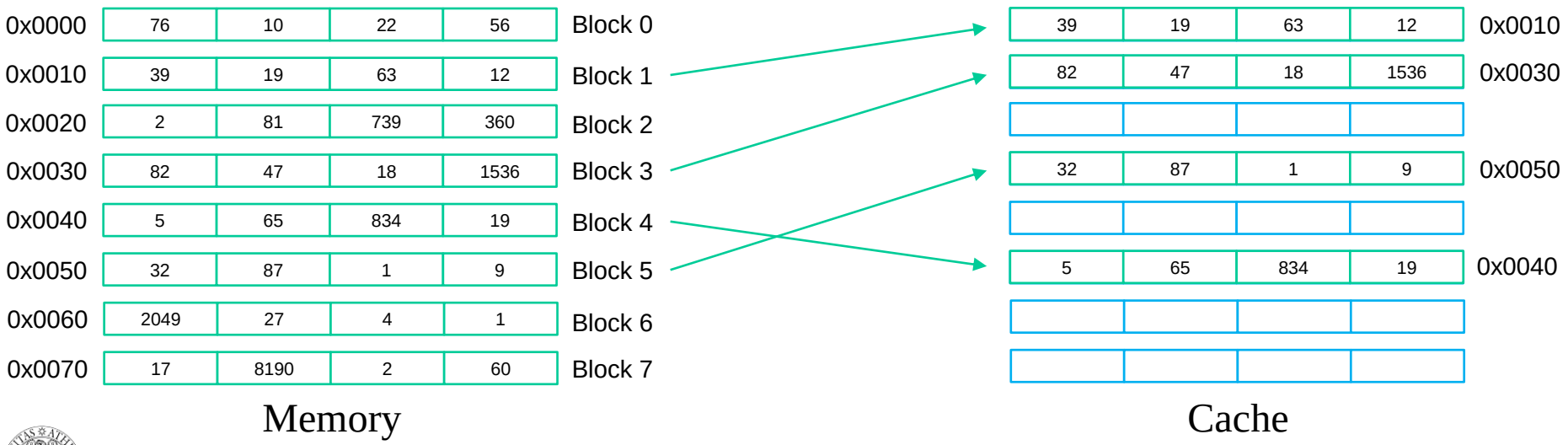
- Solution
- When always a hit
  - CPU_time = IC * CPI * $T_{clk}$ = IC * 1.0 * $T_{clk}$
- When having misses
  - CPU_time = IC * (CPI + miss_rate * mem_access / instruction * miss_penalty) * $T_{clk}$
    = IC * (1.0 + 0.02 * (1 + 0.5) * 25) * $T_{clk}$
    = IC * 1.75 * $T_{clk}$
- The "always hit" CPU is 1.75 times faster than the CPU with cache misses

# Basic cache operation

- **General structure**
  - A cache is made of a number of **blocks** that contain **words** from memory
  - The size of a block is a power of 2, to simplify addressing (e.g., a block is made of 32 words)
  - When a word is needed, **its entire block is loaded from memory** and placed *somewhere* in the cache
  - Blocks that are next to each other in memory are not necessarily stored next to each other in the cache, and vice-versa
- **A cache needs to know whether a word is stored inside it**
  - The cache needs to store a block along with its actual address in memory
  - The required address is compared to the address of all the blocks in the cache to find a match
  - If found, it is a hit, otherwise it is a miss

# Basic cache operation

- Example
  - Blocks made of 4 words, each word is 4 bytes
  - Main memory and cache organized in blocks of 16 bytes
  - If word at address 0x0010 is needed, the entire block 1 is loaded in cache
  - If word at address 0x0038 is needed, the entire block 3 is loaded in cache
  - If word at address 0x005c is needed, the entire block 5 is loaded in cache
  - If word at address 0x0054 is needed, the block is already loaded in cache
  - If word at address 0x0040 is needed, the entire block 4 is loaded in cache



# Basic problems in cache design

**The main problems in cache design are:**

1. ***Placement problem***: Where to place a block transferred from lower to higher level
2. ***Search*** (or ***identification***) ***problem***: how to determine whether the requested item is present or not
3. ***Substitution*** (or ***replacement***) ***problem***: Which block present in the cache must be replaced by one in lower-level storage, in the case of a miss
4. ***Write strategy***: what happens when a write-to-memory instructions is executed?

   **Note:** The Write strategy problem is orthogonal to the first three (once search has been completed to check whether address of write is present in cache) – write problem will be treated separately, basic architectural solutions will be discussed first

# Direct-mapped cache - 1

- Assume the cache is made of an array of $\mathbf{N_B}$ blocks
- Let $\mathbf{B_{AR}}$ be the address of the required block in RAM
  - block address is the RAM address stripped of the last bits identifying the byte within the word and the word within the block

- In a direct-mapped cache, the block can only be placed in the cache block at address $\mathbf{B_{AC}}$

$$\mathbf{B_{AC} = B_{AR} \bmod N_B}$$

- $\mathbf{B_{AC}}$ is also called the **index** of the cache block
- If $\mathbf{N_B}$ is a power of 2, then the index is equal to the $\mathbf{N_{AC} = \log_2(N_B)}$ least significant bits of $\mathbf{B_{AR}}$

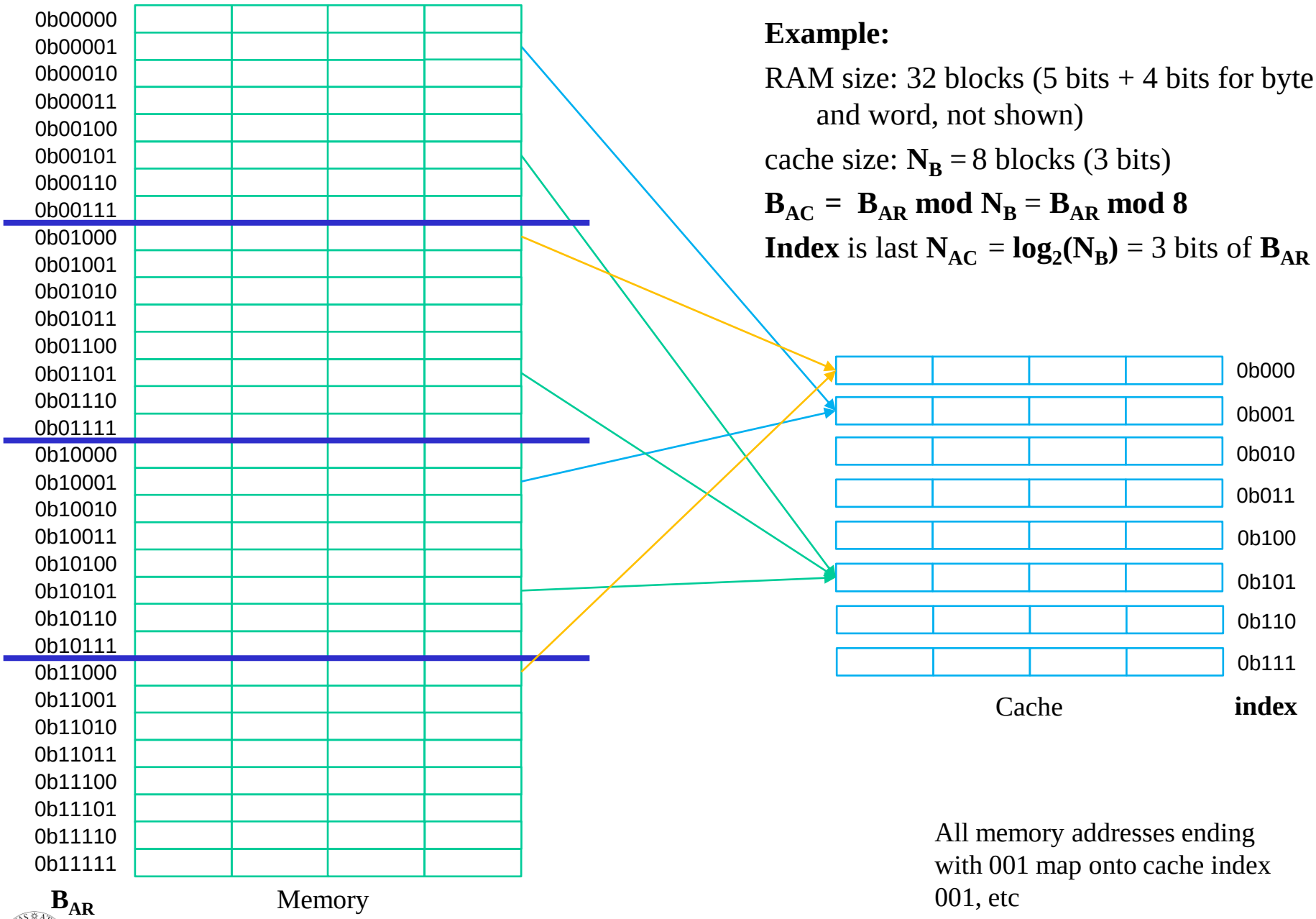
**Example:**

RAM size: 32 blocks (5 bits + 4 bits for byte and word, not shown)

cache size: $\mathbf{N_B} = 8$ blocks (3 bits)

$\mathbf{B_{AC}} = \mathbf{B_{AR}}$ **mod** $\mathbf{N_B} = \mathbf{B_{AR}}$ **mod 8**

**Index** is last $\mathbf{N_{AC}} = \mathbf{log_2(N_B)} = 3$ bits of $\mathbf{B_{AR}}$




All memory addresses ending with 001 map onto cache index 001, etc

# Direct-mapped cache - 2

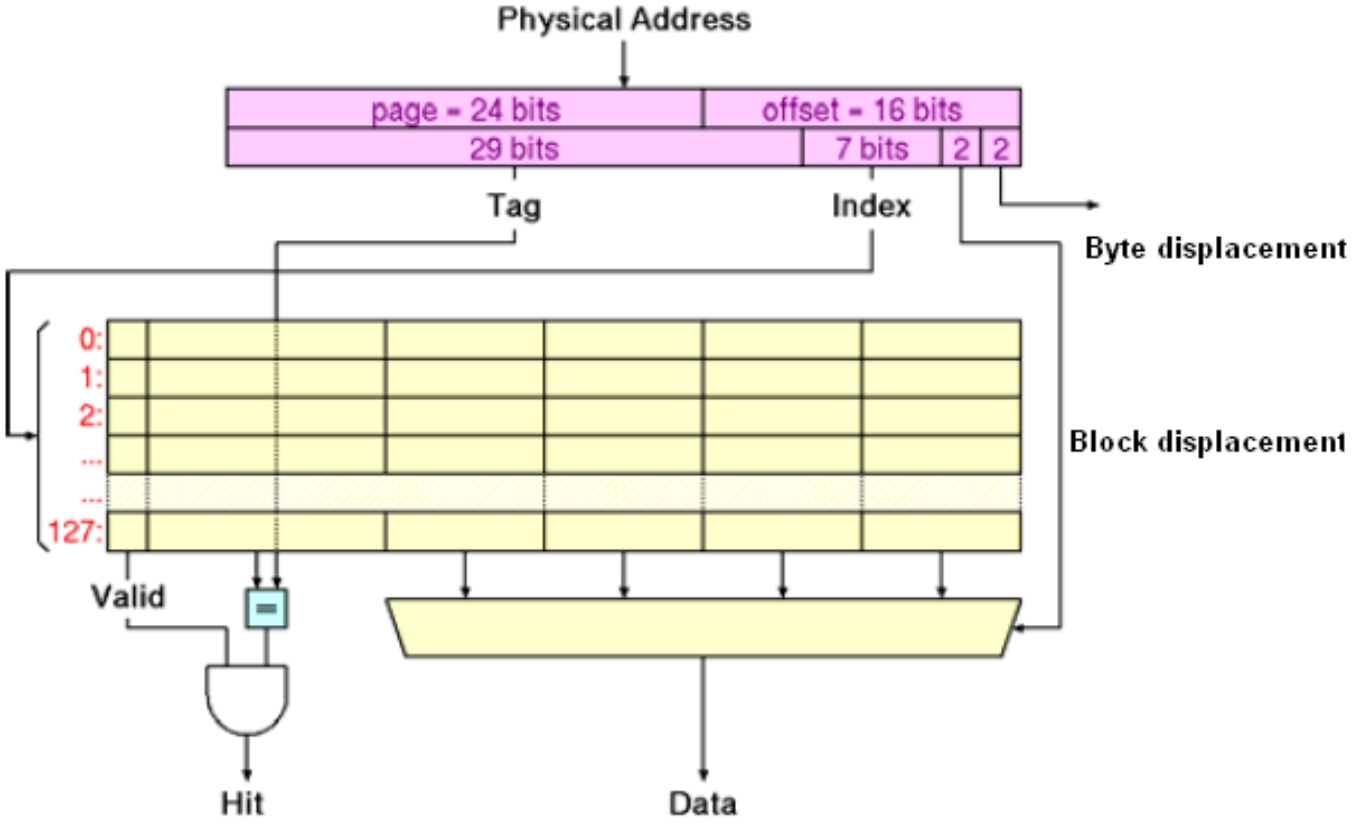
- ***Placement problem*:** Trivial – the block from RAM can be mapped onto one cache block only
- ***Search problem***: different memory blocks can be placed in the *same* cache block. How do we decide which location is mapped in the cache?
- ***Solution*:** each cache line must be provided with**:**
  - A ***tag field*** containing the $\mathbf{N_{AR}}$ **-** $\mathbf{N_{AC}}$ most significant bits of $\mathbf{B_{AR}}$

| Block address | |
|---|---|
| Tag | Index |

  - A ***validity bit*** - denotes whether the block contains meaningful (*valid*) data.
  - the presence of a given data in the cache can be verified immediately by comparing the *tag* of the line of index $\mathbf{B_{AC}}$ (if valid) with the most significant bits of the wanted address
- ***Replacement problem***: Trivial – the new block from RAM can be mapped onto one cache block only. Does not account for temporal locality! Block substituted may have been used very recently

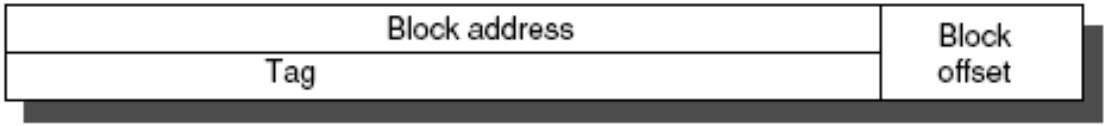
# Direct-mapped cache - 4

- Example: 40-bit memory address, 128-line cache with 128-bit block (4 words/block)
  - 2 bits for byte displacement (address of byte within a word, 4 bytes per word)
  - 2 bits for block displacement (address of word within a block, 4 words per block)
  - Index is 7 bits = $\log_2(128)$ because there are 128 blocks in the cache
  - Tag is 40 – 2 – 2 – 7 = 29 bits



# Fully associative cache - 1

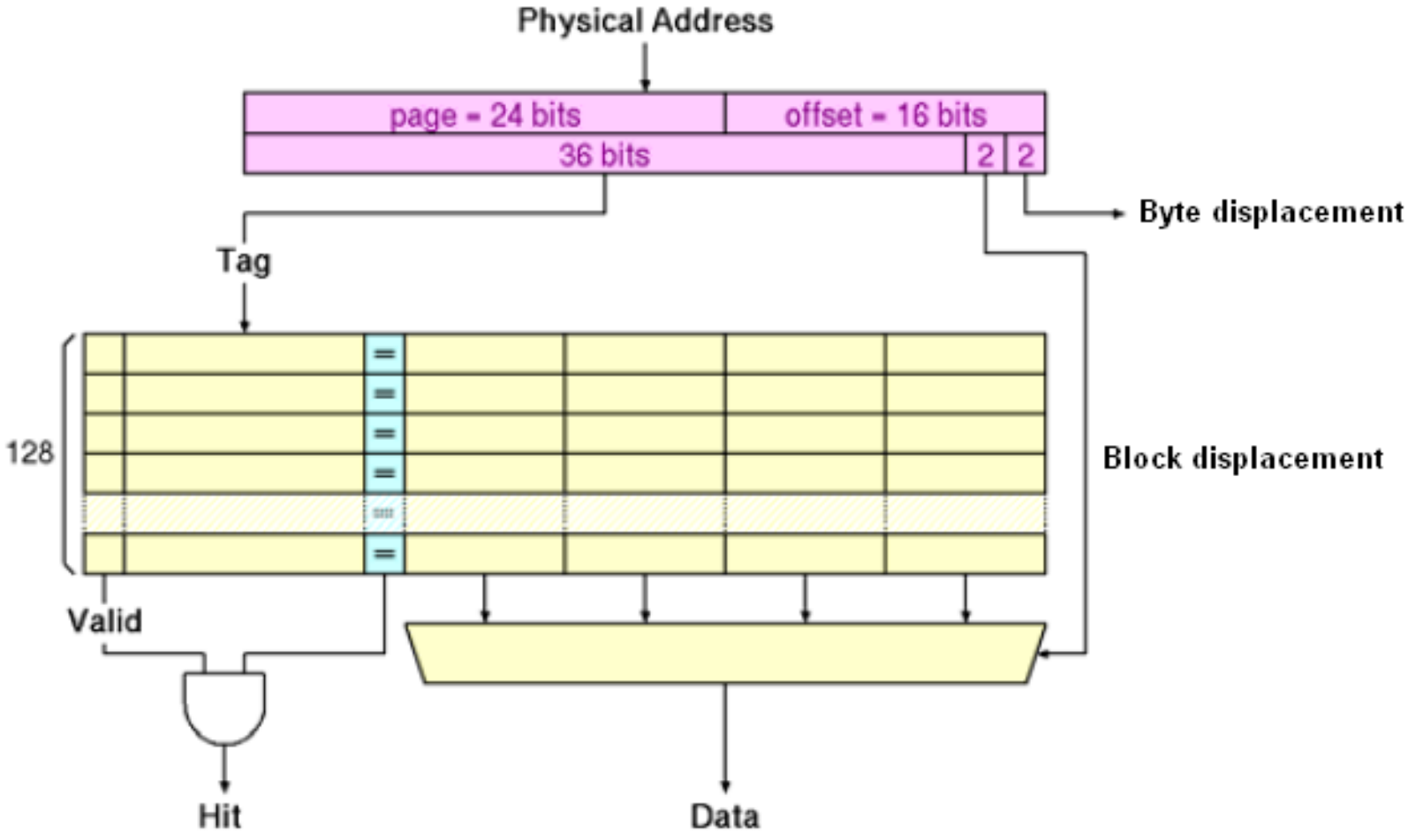
- ***Placement problem***: In a fully-associative cache every block of RAM can be mapped onto any block of cache (there is no constraint on *Placement*);
  - The **index** is 0 bits and the ***tag*** is the complete address of the word minus the bits for block and byte displacements




- ***Search problem***: The *search* problem is tackled by running a parallel comparison of the wanted address with **all tags**
  - large number of comparators → very expensive in terms of area and power consumption

# Fully associative cache - 3

**Example: 128-line cache, 128-bit block (4 words/block)**



# Fully associative cache - 2

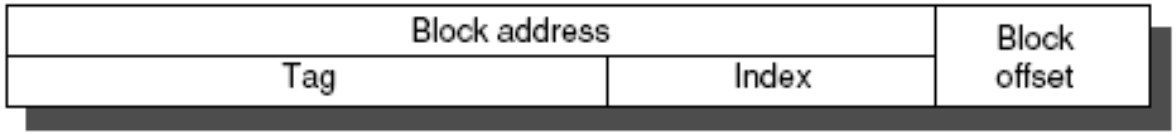
- ***Replacement problem***: it is not immediate like in DM cache. 3 possible policies:
  - ***Random*** – the block to be substituted is chosen randomly.
    - **Pros.** It is simple. Minimum HW support (pseudo-random number generator)
    - **Cons.** Efficiency in terms of miss rate is comparable to DM cache. In fact, temporal locality not taken into account.
  - ***Least Recently Used* (LRU)** – substitute the block left unused for the longest time.
    - **Pros.** It fully exploits temporal locality → maximum efficiency
    - **Cons.** It requires HW support: each block is associated with a "down" counter set to the highest value whenever the block is accessed. Whenever *a different block* is accessed, the counter is decremented by one; substitution involves block whose counter has the lowest value. The cost of this solution increases largely with the number of blocks. Actual performance *not* justified by complexity → approximations may be adopted.
  - ***First In First Out* (FIFO)** (also called ***Round Robin***) of length ***N*** – substitute the block used ***N*** accesses *before the present one* - whether it was used during the last ***N-1*** accesses or not.
    - **Pros.** This is the best tradeoff between performance and complexity. It is adopted in several recent systems.
    - **Cons.** It just approximates the locality principle

# *N*-way set-associative cache - 1

- *N*-way set-associative cache is a tradeoff between DM and fully associative solutions
- Cache is arranged in $N_{set}$ ***sets*** ($N_{set}$ being a power of 2), each set including ***N*** blocks. Notice: $N_{set} = N_B / N$ where $N_B$ is the total number of blocks
- ***Placement problem***: The index $I_{set}$ of the ***Set*** in cache corresponding to a given memory block address $B_{AR}$ is given by:

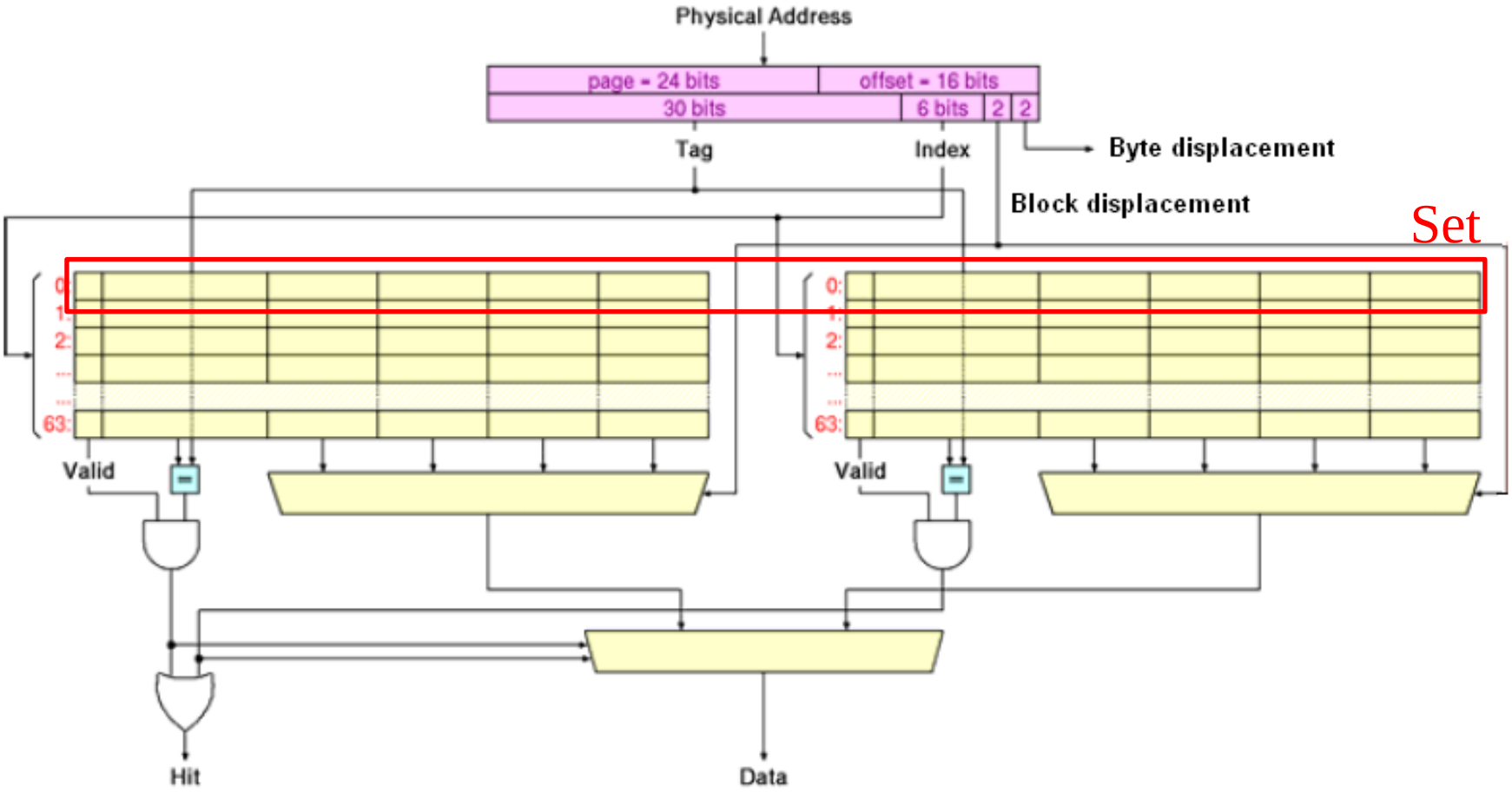
$$I_{set} = B_{AR} \bmod N_{set}$$

- Each block within the set at index $I_{set}$ can map onto any of the ***N*** cache blocks in the set, using fully associative technique
- ***Search problem***: The ***set*** in cache corresponding to block in RAM is identified as in DM cache (***Index*** identifies Set). Within the set, block is identified with associative mechanisms (multiple tag comparisons in parallel)



# *N*-way set-associative cache - 3

**Example: 2-way 128 block cache example (4 words/block)**



# *N*-way set-associative cache

- The direct mapped cache is a 1-way set associative cache (set = 1 block)
- The fully associative cache has only one big set, the size of the whole cache
- *Advantages compared to fully associative cache*: comparators are fewer and smaller → cheaper and faster solutions
- *Advantages compared to direct-mapped cache*: better exploitation of temporal locality at a reasonable cost

**Assume constant cache dimensions:**

- Increasing associativity by a factor 2:
  - Number of blocks $N$ within one set doubles → tag length one bit longer and the number of comparators doubles
  - Number of sets $\boldsymbol{N_{set}}$ is halved → Index length one bit shorter

# *N*-way set-associative cache - 4

- ***Replacement problem***: performed by one of the techniques seen for associative cache (random, LRU or round-robin). Different associativity degrees and substitution policies may affect the overall miss rate

**Example: Data cache misses per 1000 instructions in 10 SPEC2000 benchmarks**

| | Associativity | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | Two-way | | | Four-way | | | Eight-way | | |
| Size | LRU | Random | FIFO | LRU | Random | FIFO | LRU | Random | FIFO |
| 16 KB | 114.1 | 117.3 | 115.5 | 111.7 | 115.1 | 113.3 | 109.0 | 111.8 | 110.4 |
| 64 KB | 103.4 | 104.3 | 103.9 | 102.4 | 102.3 | 103.1 | 99.7 | 100.5 | 100.3 |
| 256 KB | 92.2 | 92.1 | 92.5 | 92.1 | 92.1 | 92.5 | 92.1 | 92.1 | 92.5 |

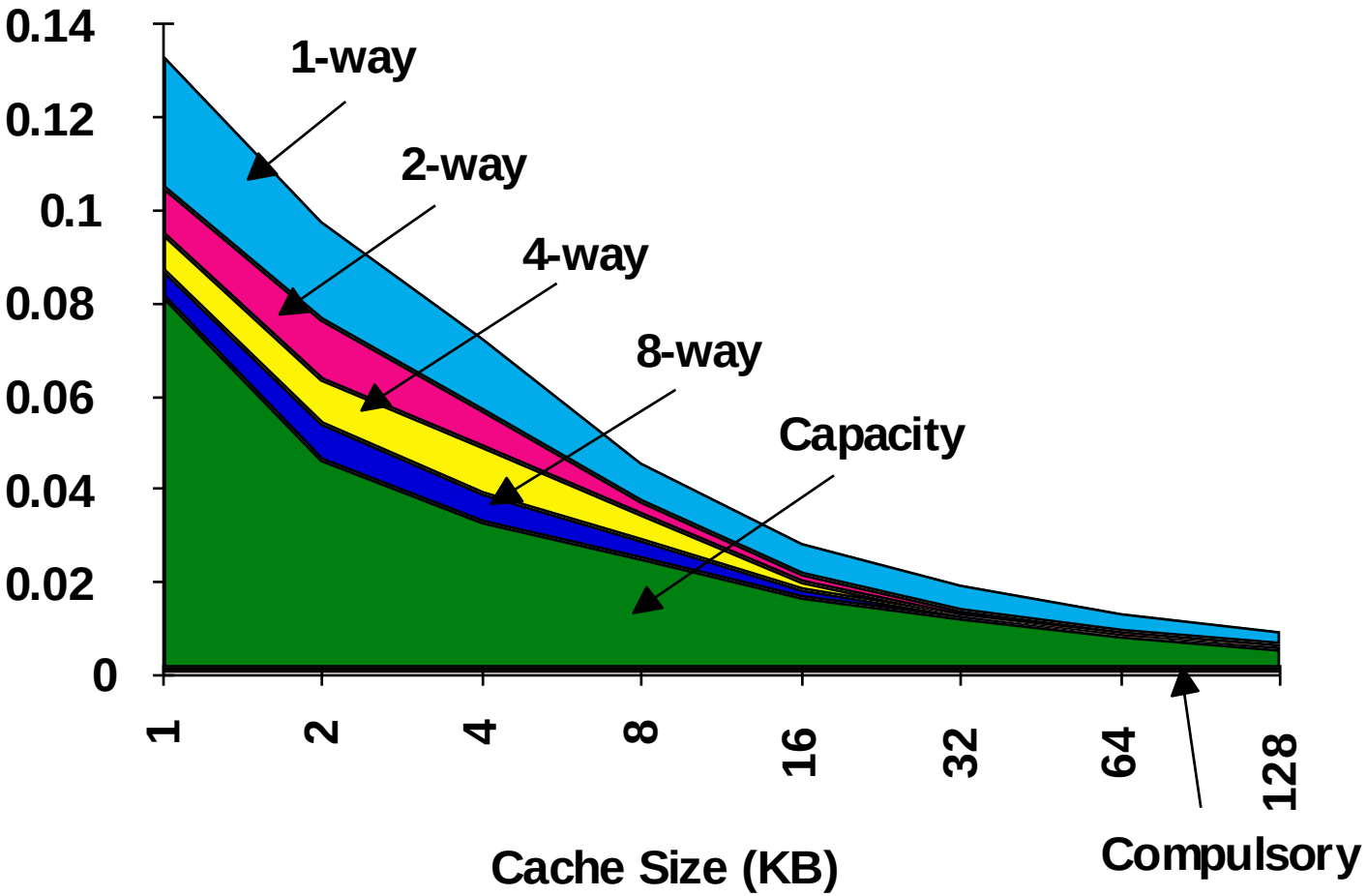
**Notes**

- There is little difference between LRU and random for the largest-size cache
- LRU outperforms the others for smaller caches.
- FIFO generally outperforms random in the smaller cache sizes.
- For large caches, the degree of associativity does not affect miss rate significantly

# Causes of cache miss

- ***Compulsory misses*** (also "cold start misses" or "first reference misses"): at the time of the first access, a block is never present in cache. Not dependent on cache size and architecture (dependent on block size, though)
- ***Capacity misses***: if the cache cannot contain all the blocks needed during execution of a program, capacity misses will occur due to blocks being discarded and later retrieved. This kind of misses decreases with cache size
- ***Conflict misses***: in direct-mapped or set-associative caches, blocks that are replaced may have to be reloaded later in the same set – leading to *collision* (*"conflict"*) misses. This kind of misses decrease with increasing cache size and associativity.
  - Usually "2:1 rule": miss rate of DM cache of *N* bytes is almost equal to the miss rate of a 2-way set-associative cache of *N*/2 bytes.
  - ***Cache thrashing***: blocks mapped on the same locations are repeatedly transferred to-from cache (worst-case of conflict misses)

# Example of miss rate contributions (SPEC92)



# The write problem

- ***Write*** in caches are less frequent than reads
  - Instruction fetch only requires memory reads
  - "Load" instructions more frequent than "store"
- Making common case fast requires optimizing performance for reads: e.g., starting block read as soon as a block address is available while performing tag verification
- In write, a block cannot be modified while the tag is checked to see if the address is a hit. Also, usually, only ***part*** of a block has to be modified by a write (usually, between 1 and 8 bytes) – suitable words must be selected within a block
- ***In Write to memory*** operations we ask:
  - ***Speed*** – implies writing ***to cache***
  - ***Consistency*** – information in lower-layer memories must always be consistent with information in cache.

# Write strategies

**Two basic policies when writing to the cache**

- ***Write-through*:** information is written ***simultaneously*** in cache block ***and*** in the main memory block
  - Consistency always respected, access time for writes is that of the lower level of memory → lower performance

- ***Write-back*** (also called ***copy-back*):** when a write instruction is executed, data are written ***only*** in a cache block**.** Lower-level memory block is modified instead ***only when a substitution occurs.***
  - Just after a write, cache and lower-level memory are ***inconsistent***: cache block may be either ***clean*** (unmodified) or ***dirty*** (modified). In this case a "dirty bit" must be added to cache. If clean, the block is not written back on a miss, since identical information to the cache is found in lower levels

- Generally, *write-back* is preferred to *write-through* because locality principle leads to high probabilities of writing again in a block recently accessed for a write → good probability of multiple writes before a substitution occurs.

# Pros and cons of write strategies

## Write back

+ Individual words are written by CPU at speed dictated by *cache*, not by lower-level memories
- Inconsistent memory
+ Multiple writes to a block involve just *one* write to lower-level memory → larger efficiency in data transfers:
  - Lower memory bandwidth
  - Lower power consumption (attractive in embedded processors)

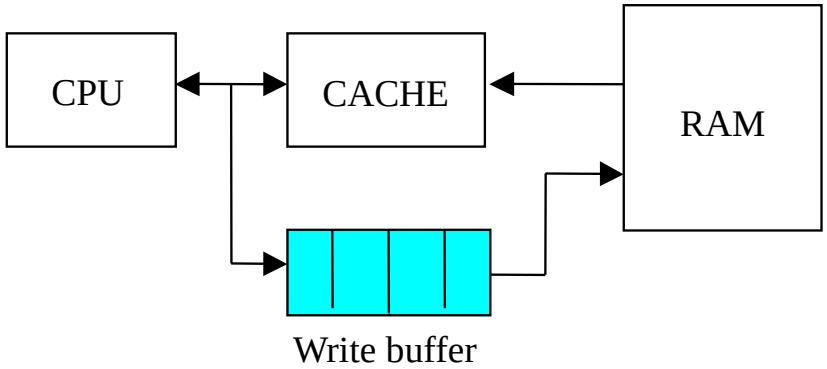
## Write through

+ Easier to implement:
  - Cache always clean
  - It does not require writing to lower memories on a read miss to replace dirty blocks
+ Perfect data consistency (preferable in I/O and multiprocessors)
- *Write stalls* in case of miss on writing (the CPU must wait for writes to complete)

# Improving the efficiency of write through

- In order to reduce the latency due to write stalls, a *write buffer* between cache and lower-layer memory is inserted
- CPU writes to cache *and* write buffer. The write buffer is a simple FIFO
- Data are transferred from CPU to buffer at cache speed, and from buffer to memory at memory speed
- Accordingly a *write stall* occurs only when the buffer is full, since so many writes may be queued as there are positions in buffer
- What happens if a read occurs from a word still in write buffer?

1. **Solution:** read miss waits for write buffer to be empty (lower performance)
2. **Solution:** on a read miss, check contents of Write Buffer with word address requested by read, if no match serve the read miss first.



# Managing write misses

- Writes can induce ***write misses***, i.e., attempts to write to an address not present in cache. Possible solutions**:**
  - ***Write allocate*** (also ***fetch on write***): the block is allocated in cache on a write miss, and then written into the lower-level memory (according to the chosen write policy)
  - ***No write allocate*** (also ***write around***): the block is modified directly in lower-level memory, without being transferred to cache

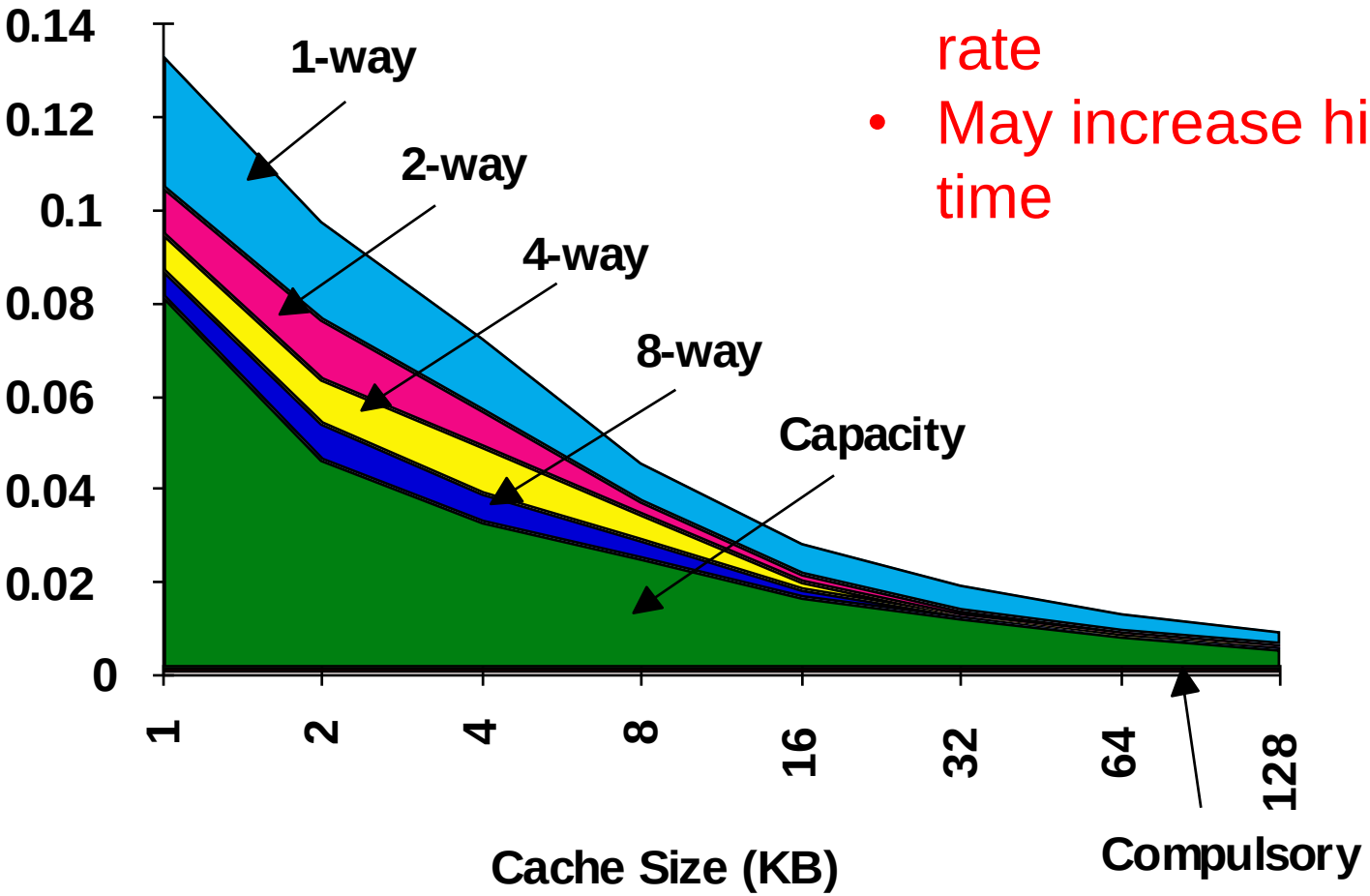
- In general**:**
  - ***Write back*** caches tend to use ***write allocate*** option (due to locality principle, it is hoped that subsequent writes will involve the same block)
  - ***Write through*** caches tend to use ***no write allocate*** (writes must anyway go to lower-level memory, no gain in passing through cache)

# Improving cache performance

Aspects to consider

- **Miss rate** (compulsory, capacity, conflict)
- **Miss penalty**
- **Hit time**

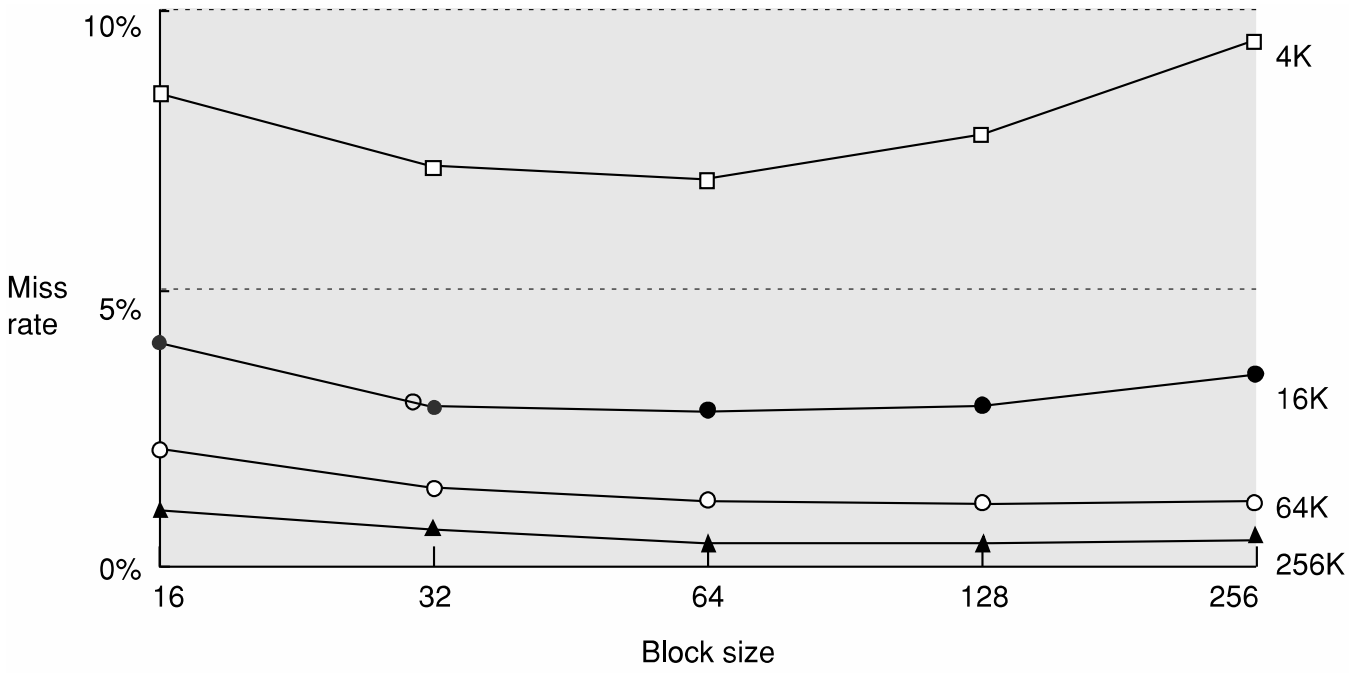
# Increase cache size




- Decreases miss rate
- May increase hit time

# Increase block size

- Larger block size **reduces compulsory misses** (because of spatial locality)
- Larger block size **increases miss penalty** (larger block to read from memory)
- Larger block size implies fewer blocks for a fixed cache size, which may **increase conflict misses and capacity misses** (if cache is small)



# Larger block size to reduce miss rate

- Assume memory takes 80 clock cycles overhead, and then delivers 16 bytes every 2 clock cycles
  - Hence, 16 bytes in 82 cc, 32 in 84, and so on
  - **AMAT = hit_time + miss_rate * miss_penalty**

| | Cache size | | | |
|---|---|---|---|---|
| **Block size** | **4K** | **16K** | **64K** | **256K** |
| 16 | 8.57% | 3.94% | 2.04% | 1.09% |
| 32 | 7.24% | 2.87% | 1.35% | 0.70% |
| 64 | 7.00% | 2.64% | 1.06% | 0.51% |
| 128 | 7.78% | 2.77% | 1.02% | 0.49% |
| 256 | 9.51% | 3.29% | 1.15% | 0.49% |

miss rate

| | | Cache size | | | |
|---|---|---|---|---|---|
| **Block size** | **Miss penalty** | **4K** | **16K** | **64K** | **256K** |
| 16 | 82 | 8.027 | 4.231 | 2.673 | 1.894 |
| 32 | 84 | **7.082** | 3.411 | 2.134 | 1.588 |
| 64 | 88 | 7.160 | **3.323** | **1.933** | **1.449** |
| 128 | 96 | 8.469 | 3.659 | 1.979 | 1.470 |
| 256 | 112 | 11.651 | 4.685 | 2.288 | 1.549 |

AMAT

# Split cache vs. unified cache

- ***1st level cache***: choice between separate I- and D- cache or unified cache
- "Separate" caches can be optimized autonomously – statistics prove that instruction caches have lower miss rate than data caches. In fact:
  - Instruction cache: "read-mostly" – very good spatial locality (except in the case of very frequent procedure calls);
  - Data cache: locality strongly application-dependent.

**Table: values obtained using SPEC benchmarks**

| Size | I-cache | D-cache | Unified |
|---|---|---|---|
| 4 KB | 1.78% | 15.94% | 7,24% |
| 16 KB | 0.64% | 6.47% | 2.87% |
| 64 KB | 0.15% | 3.77% | 1.35% |
| 128 KB | 0.02% | 2.88% | 0.95% |

# Split cache vs. unified cache

- Average Memory Access Time for split caches

  miss_rate_I = miss rate for instruction cache
  miss_rate_D = miss rate for data cache

  **AMAT** = %Instr_references (hit_time + miss_rate_I * miss_penalty ) +
  %Data_references (hit_time + miss_rate_D * miss_penalty)

- Note:
  %Instr_references = fraction of total memory references for instructions
  %Data_references = fraction of total memory references for data

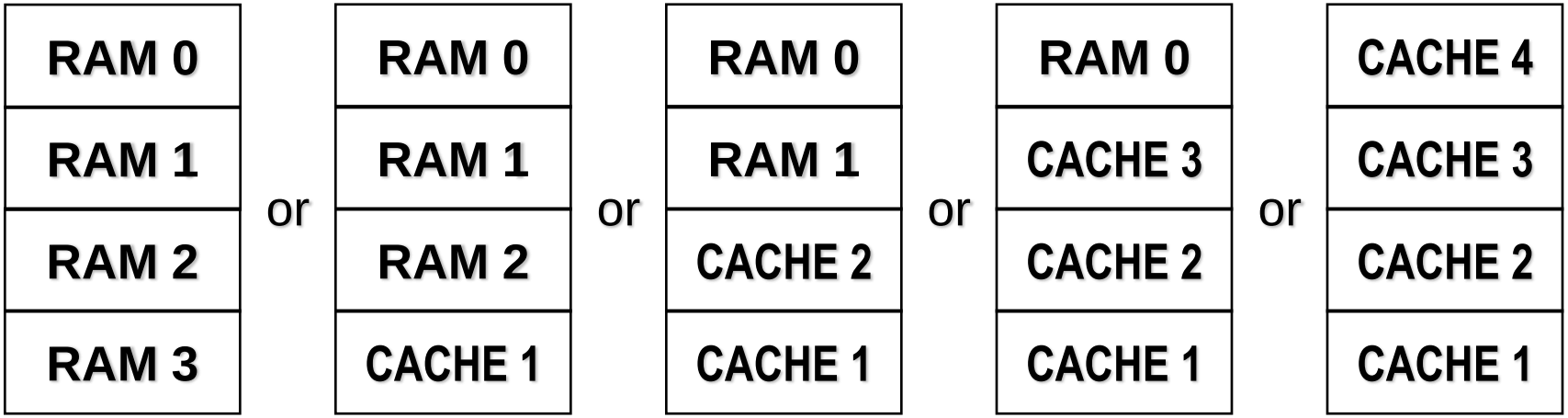
  This is **not** the same as the fraction of loads/store over the total number of instructions

# Cache locking

- Thrashing strongly reduces performance: application-specific embedded systems allow to identify blocks of data or instructions that might be subject to thrashing

- ***Cache locking***: some sections of cache can be ***locked*** by the programmer to be used as fast local memories. In this case:
  - The locked portion of cache is visible to the programmer
  - The caching hardware of the locked section (e.g., tags, replacement logic) is turned off (i.e., cache becomes a normal RAM). This is possible only with associative or set-associative caches.

**Example. The 64kB 4-way set-associative cache on TI C6711 DSP can be configured as follows:**



# Multi-level cache - 1

- Cache speed decreases with size
- Miss penalty depends on speed of lower-level memory
- Want large but fast, and low miss penalty
  - **Insert a slower but larger second-level cache between the first level cache and main memory**
- First-level cache is small enough to be very fast (speed close to CPU)
- Second-level cache large enough to capture many of the accesses that would go to main RAM in case of misses
- Also the second-level cache is transparent to programmer and compiler
  - Transfers to/from managed by hardware

# Multi-level cache - 1

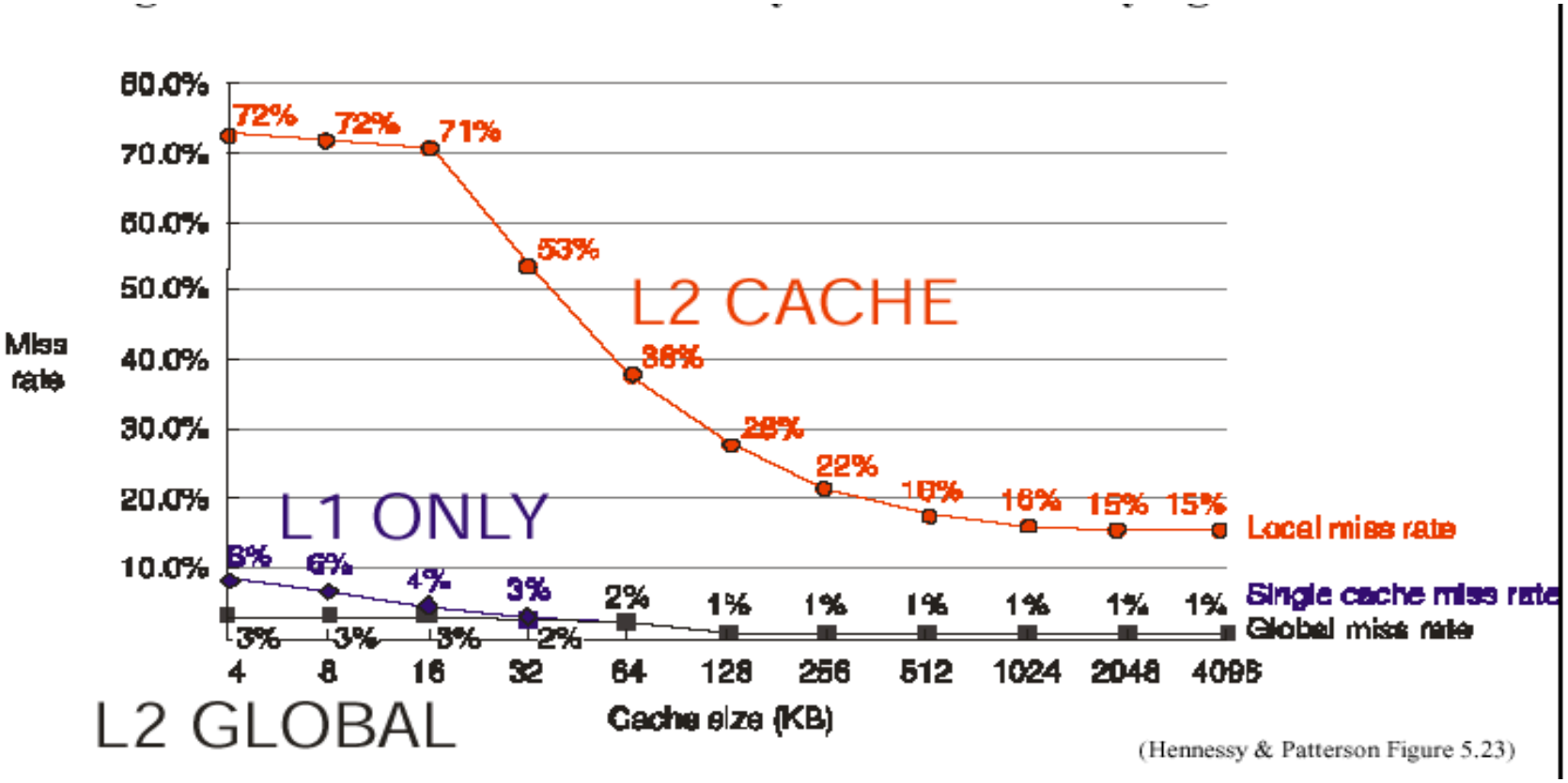
- The total AMAT is given by:

$$T_A = Hit_time_{L1} + Miss_rate_{L1} * Miss_Penalty_{L1}$$

$$= Hit_time_{L1} + Miss_rate_{L1} * (Hit_time_{L2} + Miss_rate_{L2} * Miss_penalty_{L2})$$

- There are now two kinds of miss rates:
  - ***Local miss rate***: number of misses in a cache divided by the total number of accesses to *that* cache ($Miss_rate_{L1}$ for L1 and $Miss_rate_{L2}$ for L2)
  - ***Global miss rate***: number of misses in a cache divided by total number of memory accesses generated by CPU (either $Miss_rate_{L1}$ for L1 and $Miss_rate_{L1} * Miss_rate_{L2}$ for L2)

# Multi-level cache - 2




(Hennessy & Patterson Figure 5.23)

**Observations**

- Second-level cache makes sense if it is much larger than L1 caches
- Global miss rate close to single-cache L1 local miss rate
- Speed of L1 cache affects CPU clock rate, speed of L2 cache only affects miss penalty of L1 cache

# Multi-level cache - 3

- L1 and L2 caches should be different to improve overall performance.

**Issues**

1. *Multilevel inclusion vs. Multilevel exclusion*
2. *Split vs. Unified & bandwidth vs. flexibility*
3. *Write through vs. write back & write allocation*
4. *Block size & latency vs. bandwidth*
5. *Associativity vs. cycle time*

1. *Question*: *Should data in L1 be present in L2 as well?* Two policies:

   - ***Multilevel inclusion***: L1 data always present in L2 (it ensures consistency between I/O and caches). Problem: performance analysis suggests that L1 blocks may be smaller than L2 ones so that substitution might become difficult (**Example:** in Pentium IV L1 has 64-byte blocks and L2 has 128-byte blocks)
   - ***Multilevel exclusion***: L1 data are *never* found in L2. Reasonable if L1 and L2 have similar size (**Example:** AMD Athlon has 2 64 kB L1 cache and 1 256 kB L2 cache)

# Multi-level cache - 4

2. *Question: should I use a split or a unified cache?*

- Performance analysis showed that the best approach is to use
- **L1 cache: Split**
  - High bandwidth for simultaneous data and instruction access
  - Separate optimization
- **L2 cache: Unified**
  - Reduced area size
  - Better flexibility on memory usage

3. *Question: what write policy is better for each level?*

- **L1 cache: Write through + no-write-allocate**
  - Avoids problems with coherence in L1
  - Simpler implementation for on-chip solution
- **L2 cache: Write back + write-allocate**
  - Reduces overall bus traffic
  - Captures temporal and spatial locality

# Multi-level cache - 5

4. *Question: what is the best block size?*
   - A larger block size assures a better use of spatial locality
   - Compulsory misses decrease
   - Conflict and capacity misses may increase, since larger blocks means fewer blocks in cache
   - However, miss penalty increases ⇨ higher access time

- Statistics show optimum block size between 32 bytes (for small L1 caches) and 64 bytes (for large L2 caches).

5. *Question: what degree of associativity for each level?*

- **L1 cache: No clear winner**
  - DM: faster cycle time, lower hit rate
  - Set associative: longer hit-time vs. better hit rate
- **L2 cache: No clear winner**
  - Set-associative less advantageous for really large caches
  - Set-associative L2 gives flexibility

# Summary: L1 vs. L2 tradeoff examples

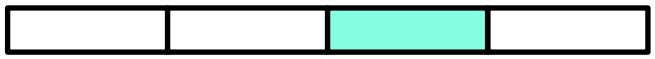
| **Pentium Pro** | **L1** | **L2** |
|---|---|---|
| Size | 16KB | none - 256KB - 512KB |
| Organization | Split (8KB + 8KB) | Unified |
| Write Policies | programmable | programmable |
| Block size | 32 bytes | 32 bytes |
| Associativity | D: 2-way; I: 4-way | 4-way |

| **MIPS R10000** | **L1** | **L2** |
|---|---|---|
| Size | 64KB | 512KB - 16 MB |
| Organization | Split (32KB + 32KB) | Unified |
| Write Policies | write back | write back |
| Block size | D: 32 bytes I: 64 bytes | 64 or 128 bytes |
| Associativity | 2-way | 2-way |

# Other improvements: critical word first, early restart

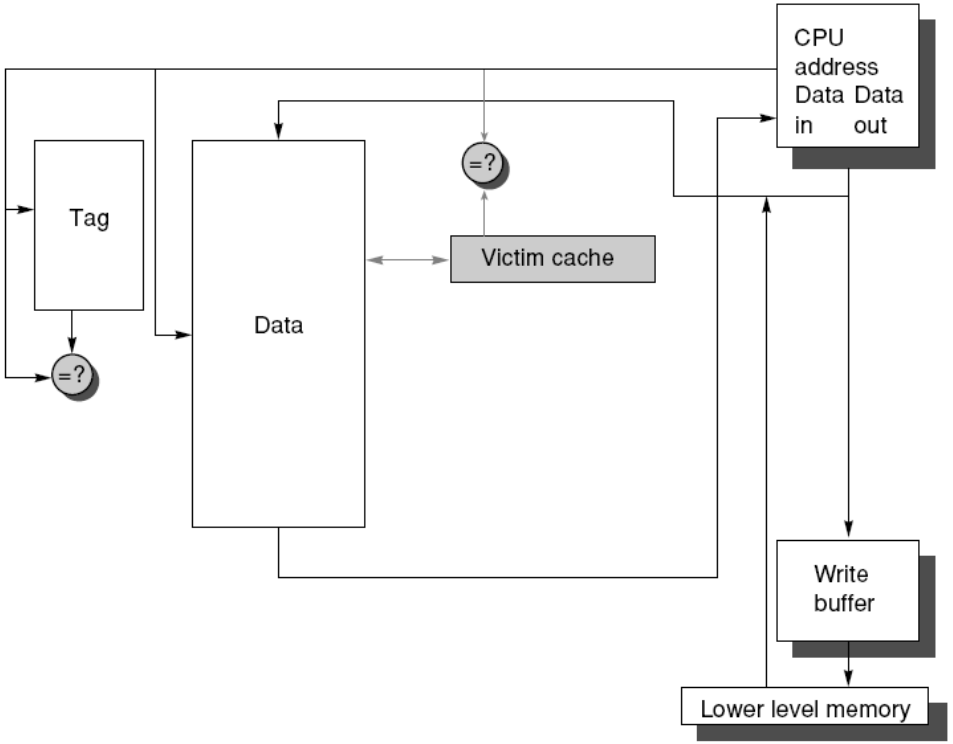
Improvements based on the observation that CPU normally needs just one word of the block at a time

- *Critical Word First:* request missed word from memory, send it to CPU as soon as it arrives; CPU runs while the rest of the cache block is filled (also called *wrapped fetch*, *request word first*)
- *Early restart*: words are fetched in normal order, as soon as the requested one arrives it is sent to the CPU
- Useful only if blocks are large



# Other improvements: victim cache

- **Idea:** what has been discarded might be used again soon.
- A small, fully associative cache (*victim* cache) can be inserted between cache and its refill path; Victim cache contains only blocks discarded from cache on a miss (*victims*) that are checked on a miss before going to lower-level memory.
- If requested data are in victim cache, victim block and cache block are swapped



# Other improvements

- **Compiler optimizations:** Compiler can increase locality by properly arranging data in memory
- Very effective both on instructions and data, compiler must be aware of the cache architecture

- **Prefetching**: start loading data in cache *before* it is referenced
- Loading a block is a simple implementation of prefetching
- Prefetching can be controlled by hardware using a stream buffer
- Prefetching can be controlled by software using special prefetch instructions introduced by the compiler

# Dissecting a cache

- **Figure out what cache your computer uses by running software!**

```
// Repeat to check caches of different sizes
for ( csize = CACHE_MIN; csize <= CACHE_MAX; csize = csize * 2 ) {
  printf( "----------\n" );
  // We access the data in strides. Initially the stride is 1. Then we increase the stride to access every other element, and so on.
  for ( stride = 1; stride <= csize / 2; stride = stride * 2 ) {
    // Initialize the time required to access the data
    sec = 0;
    // We scan the array of data up to the selected cache size (adjusted for the stride)
    limit = csize - stride + 1;
    // We count the number of steps, so that we can repeat them later
    steps = 0;
    // Now scan the array for 1 second
    do {
      // Record the starting time
      sec0 = get_seconds( );
      // Repeat the access 10 times times the stride
      for ( i = SAMPLE * stride; i != 0; i = i - 1 ) {
        for ( index = 0; index < limit; index = index + stride ) {
          // Access the value in the array
          x[ index ] = x[ index ] + 1;
        }
      }
      steps = steps + 1;
      sec = sec + ( get_seconds( ) - sec0 );
    } while ( sec < 1.0 );
```

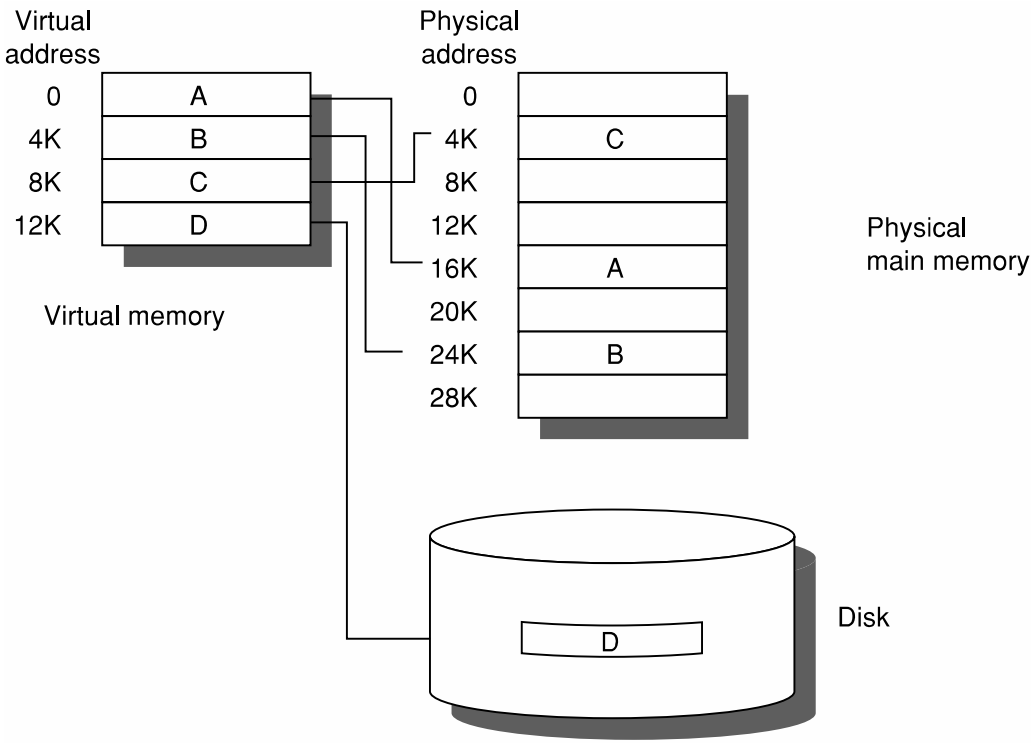
# Virtual memory

# Virtual memory

- Computers run multiple processes, each with its own address space
- Can’t give each process a full address space worth of memory
  - Processes must share a smaller amount of processor memory
- **Solution:** divide physical memory into blocks, allocate them to different processes
- Need a *protection scheme* so that processes can access only their blocks
- Virtual memory also useful to offload to secondary storage (disks) parts of (block) programs and data that can’t fit in main memory
  - The swapping between main memory and secondary storage is automatically managed by the operating system

# Virtual memory

- Process sees a virtual address space
- Blocks in virtual space are allocated anywhere in physical memory or in secondary storage
- The technique also manages relocation



# Virtual memory

- Similar to caches
  - Blocks usually called *pages* or *segments*
  - Misses are called *page faults*
- The virtual address generated by the process must be translated into the physical address
  - Called *memory mapping* or *address translation*

| Parameter | First-level cache | Virtual memory |
|---|---|---|
| Block (page) size | 16–128 bytes | 4096–65,536 bytes |
| Hit time | 1–3 clock cycles | 100–200 clock cycles |
| Miss penalty | 8–200 clock cycles | 1,000,000–10,000,000 clock cycles |
| (access time) | (6–160 clock cycles) | (800,000–8,000,000 clock cycles) |
| (transfer time) | (2–40 clock cycles) | (200,000–2,000,000 clock cycles) |
| Miss rate | 0.1–10% | 0.00001–0.001% |
| Address mapping | 25–45 bit physical address to 14–20 bit cache address | 32–64 bit virtual address to 25–45 bit physical address |

# Differences with caches

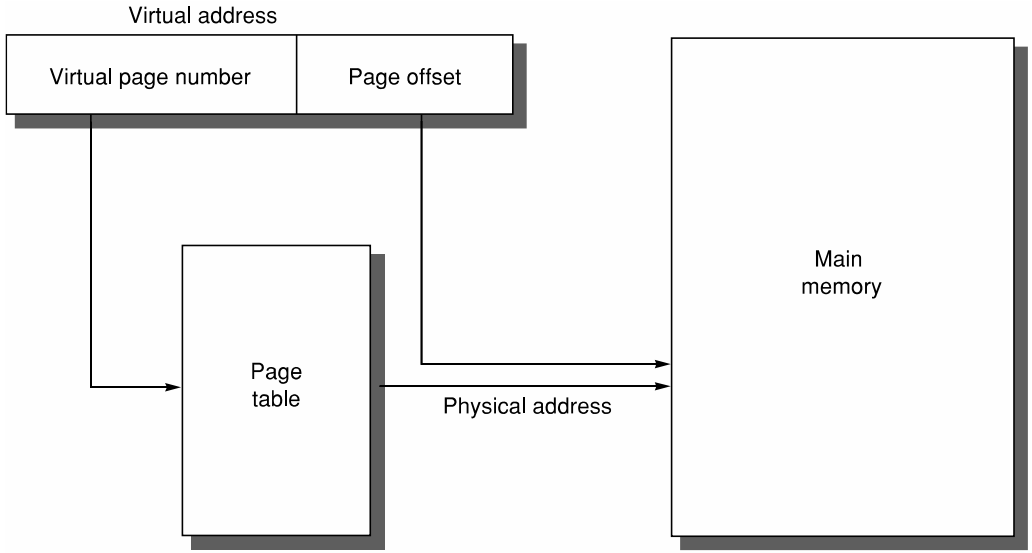
- Replacement
  - Caches: controlled by the hardware
  - Virtual memory: controlled by the operating system. The high miss penalty means it is more important to make a good replacement!
- Size
  - Processor address bus size determines size of virtual memory
  - Cache size independent of processor address bus size
- Secondary storage
  - Also used for file system

# Q1: Placement, Q3: Replacement, Q4: write

- Where can a block be placed in main memory?
  - Must reduce miss rate, given the high miss penalty
  - Hence, blocks can be placed anywhere in main memory
  - It is like a **fully associative** cache
- Which block should be replaced on a page fault?
  - Need to minimize number of page faults
  - Most operating systems use **LRU**
- What happens on a write?
  - Disks take millions of clock cycles to access
  - All virtual memory systems use **write-back**!

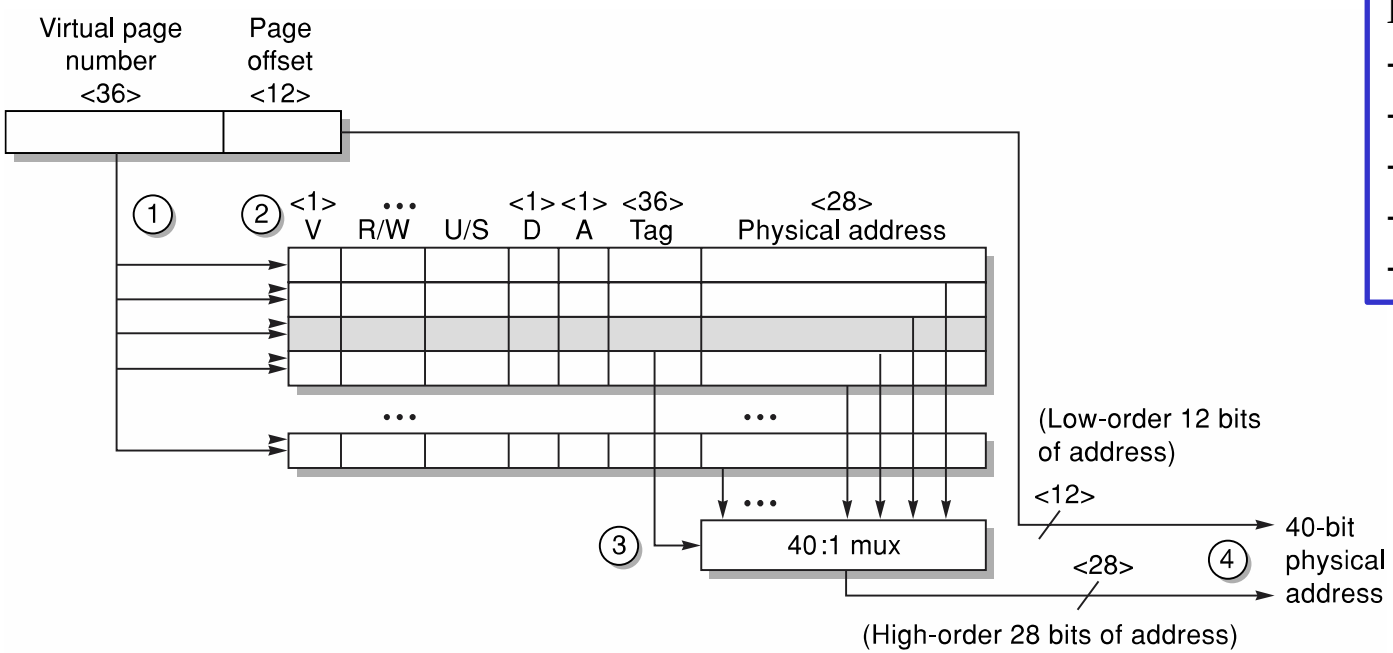
# Q2: Search

- How is a block found if it is in main memory?
- **Solution**: use a table to record correspondence between virtual page number and physical address
  - How large is the table? -- 32-bit virtual address, 4 kB page size, 4 bytes per page table entry: $2^{32} / 2^{12} * 4 = 4$ Mbytes
  - Sometimes a hash table is used, the size of the actual number of pages in physical memory in use



# Fast address translation

- Page tables are usually large
  - stored in main memory and paged!
  - implies 2 memory accesses: one for translation, one for data
- Address translation uses a special fully associative cache
  - *Translation lookaside buffer* (TLB)



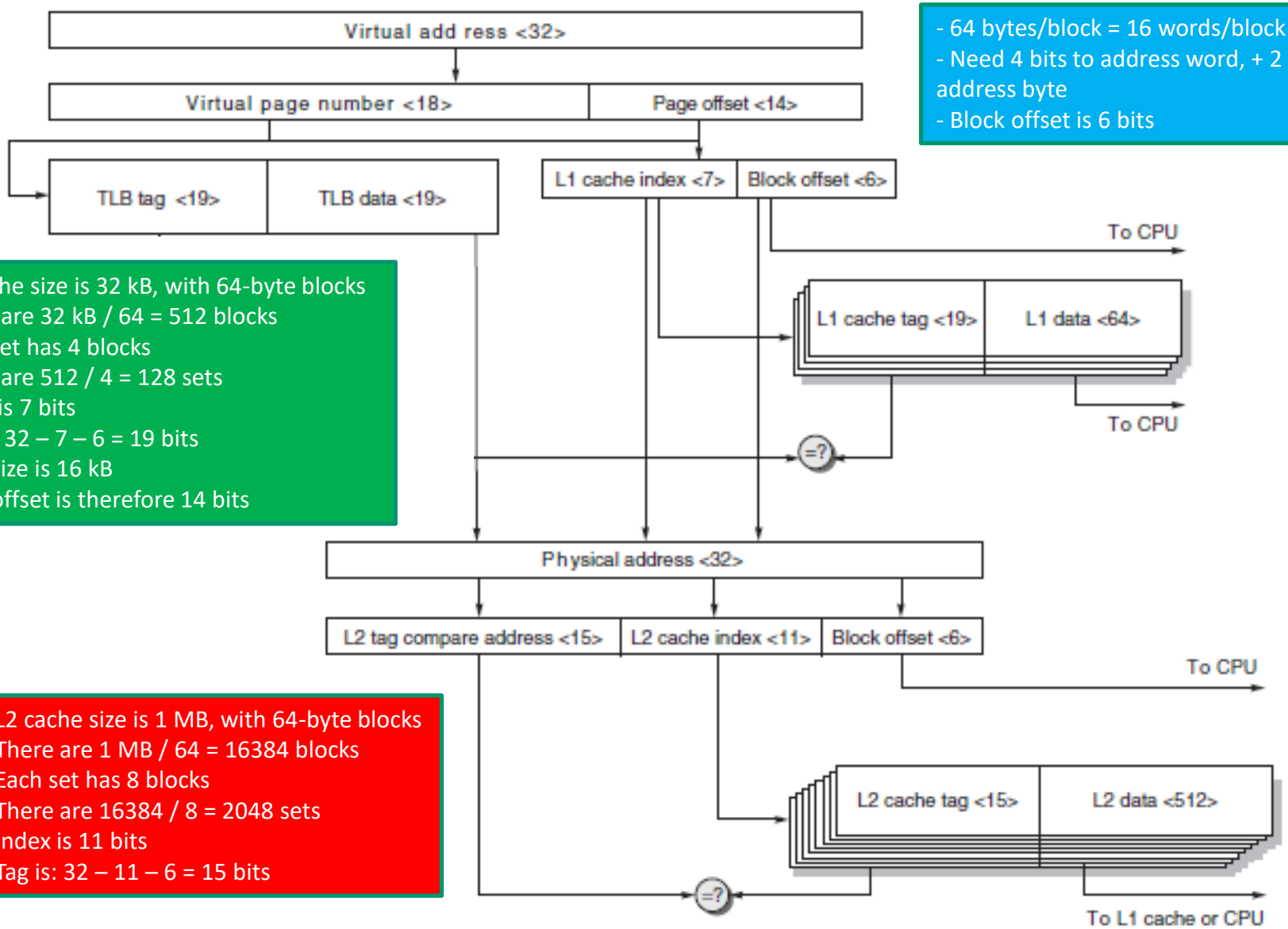

Bits:
- V: valid
- R/W: read/write
- U/S: user/system
- D: dirty
- A: accessed

# ARM Cortex-A8

- Used in the iPad, iPhone4 (together with a GPU)
- Double issue up to 1 GHz
- Split L1 cache, unified L2 cache
- The TLB is fully associative with 32 entries
  - Round robin replacement
  - Variable page size: 4 KB, 16 KB, 64 KB, 1 MB, 16 MB
- The L1 cache is four-way set associative with 64-byte blocks and 32 kB capacity, with random replacement
  - **Virtually indexed, physically tagged**
  - Miss penalty for 1 GHz is 11 clock cycles
- The L2 cache is eight-way set associative with 64-byte blocks and 1 MB capacity
  - **Physically indexed, physically tagged**
  - Miss penalty is 60 clock cycles

# ARM Cortex-A8 (16 kB page size)

- 64 bytes/block = 16 words/block
- Need 4 bits to address word, + 2 bits to address byte
- Block offset is 6 bits



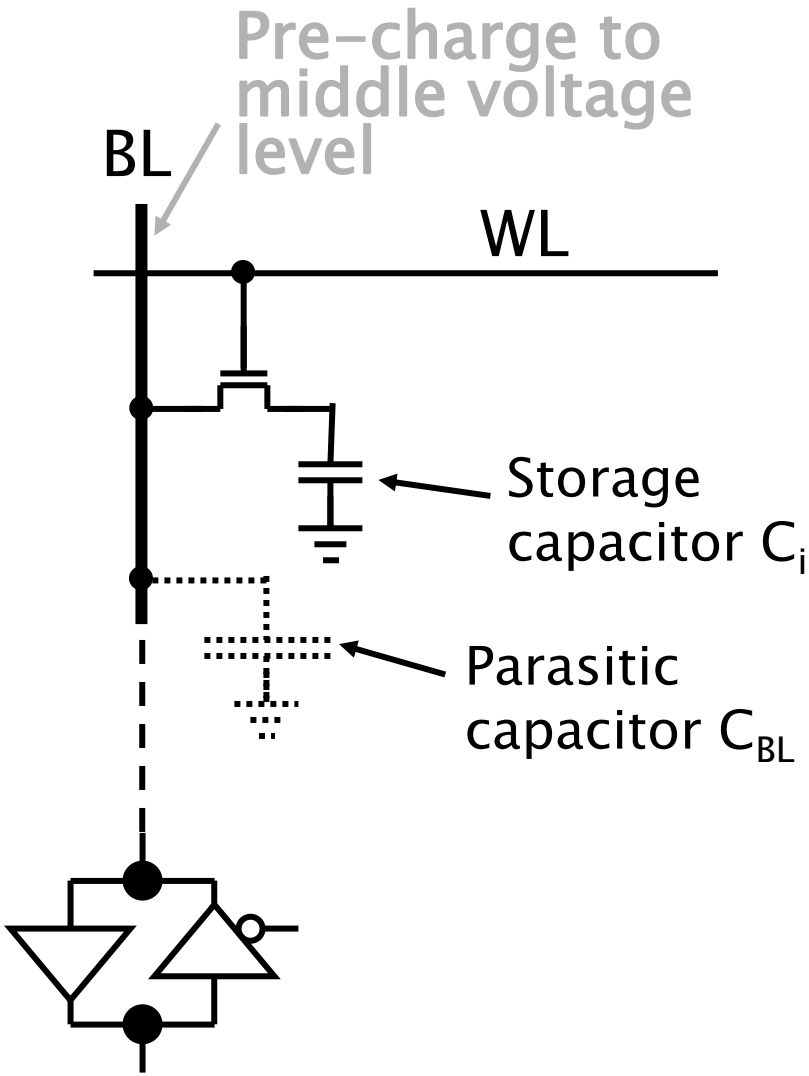

- L1 cache size is 32 kB, with 64-byte blocks
- There are 32 kB / 64 = 512 blocks
- Each set has 4 blocks
- There are 512 / 4 = 128 sets
- Index is 7 bits
- Tag is: 32 – 7 – 6 = 19 bits
- Page size is 16 kB
- Page offset is therefore 14 bits

- L2 cache size is 1 MB, with 64-byte blocks
- There are 1 MB / 64 = 16384 blocks
- Each set has 8 blocks
- There are 16384 / 8 = 2048 sets
- Index is 11 bits
- Tag is: 32 – 11 – 6 = 15 bits

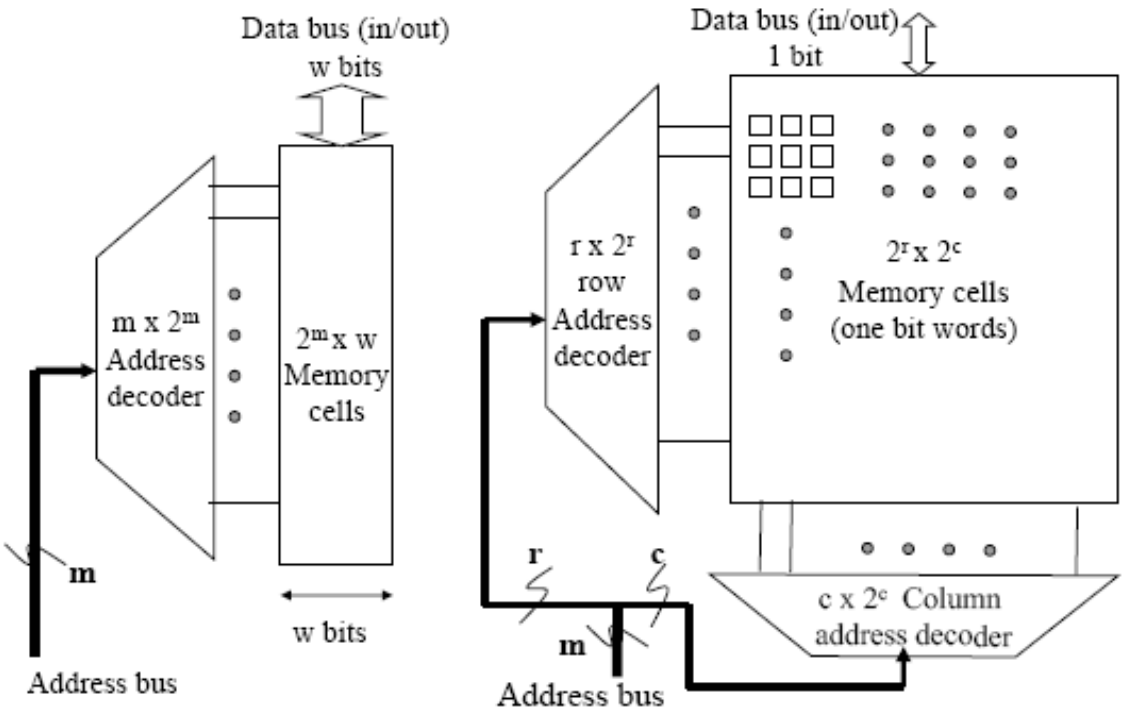
# The “main” memory

# A Dynamic RAM (DRAM) cell - 1

- In most dynamic memories 1 cell consists of just 1-transistor as a storage capacitor $C_i$
- Charge sharing with bit line capacitance ($C_i << C_{BL}$)
- Due to leakage currents, memory cell content must be refreshed periodically
- Reading much slower than writing because it implies 3 operations:
  - Precharge
  - Reading
  - Write back
- DRAM much slower than SRAM and it is also more power hungry



# "Typical" memory structure



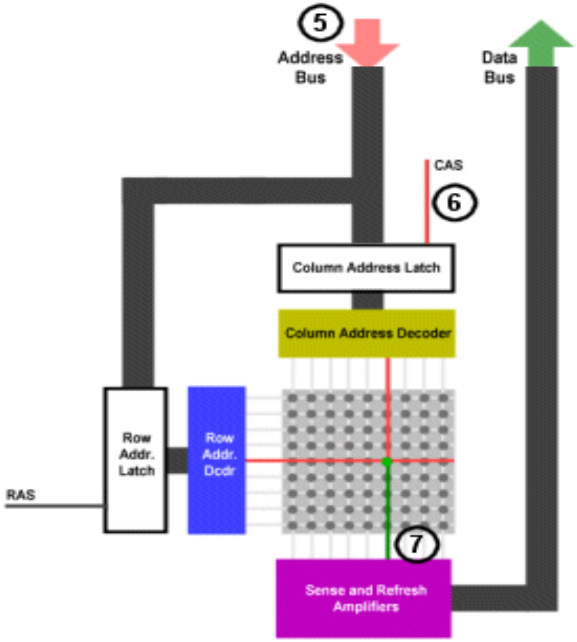

(a) (b)

- The SRAM memory has usually a cell array layout
- Ideally it should be linear (a), but this approach is not suitable
  - Long data bus→ large capacity
  - Address decoder will have a lot of logic levels
  - Memory layout too long and thin
- It is better to split the *m*-bit address into an *r*-bit row address and a *c*-bit column address (b) → smaller decoders, almost square memory layout, faster architecture (lower WL and BL delays)

# DDR commands

- PRECHARGE
  - Close current row, get ready for an activate
- ACTIVATE (RAS)
  - Open a row and store row result in a register
  - Row is open until next precharge command
- READ (CAS)
  - Read values from open row
- WRITE (CAS)
  - Write values to open row
- REFRESH
  - Restore values in rows (essentially a periodic ACTIVATE)

# “Typical” DRAM memory structure

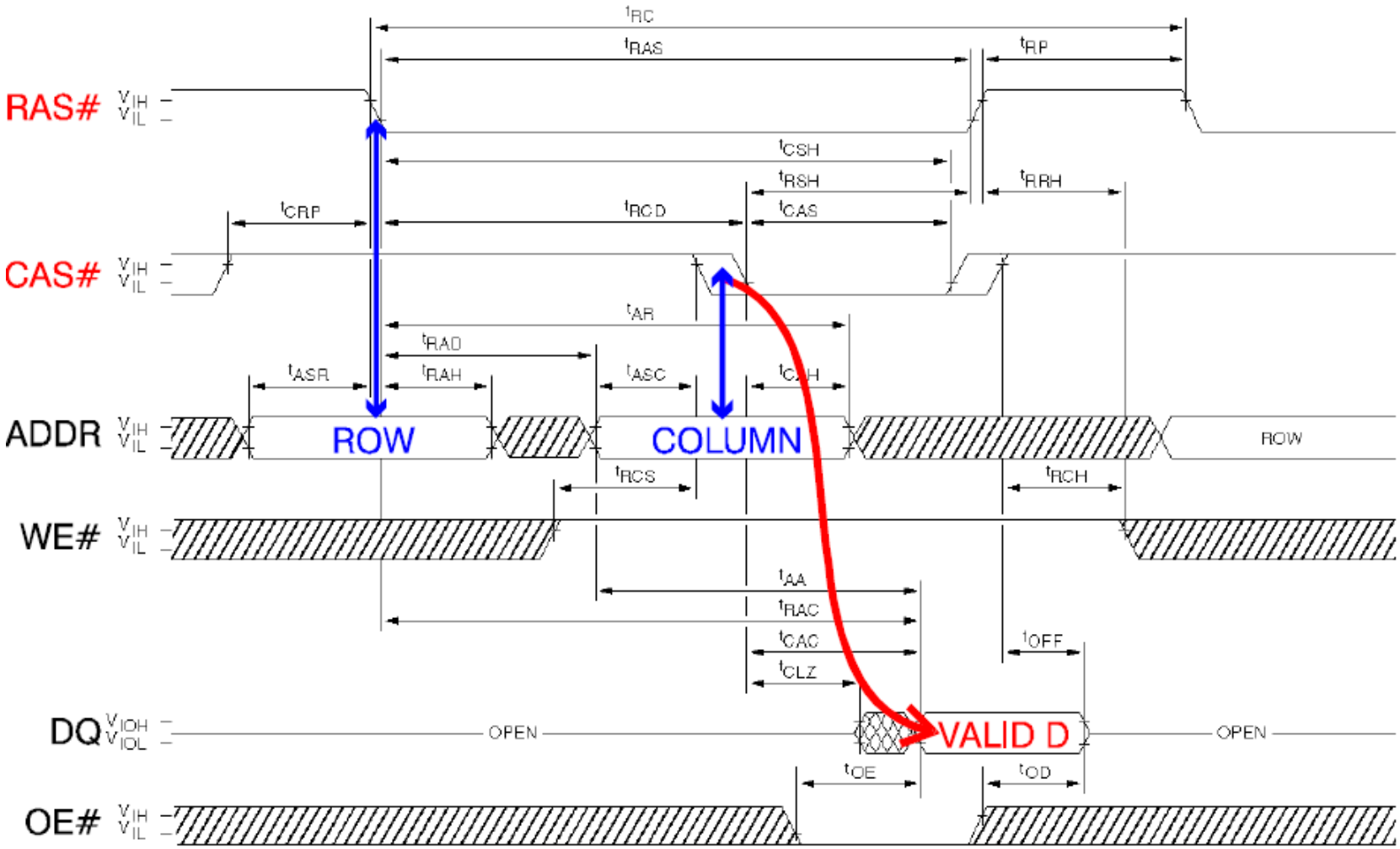


**Memory chip**

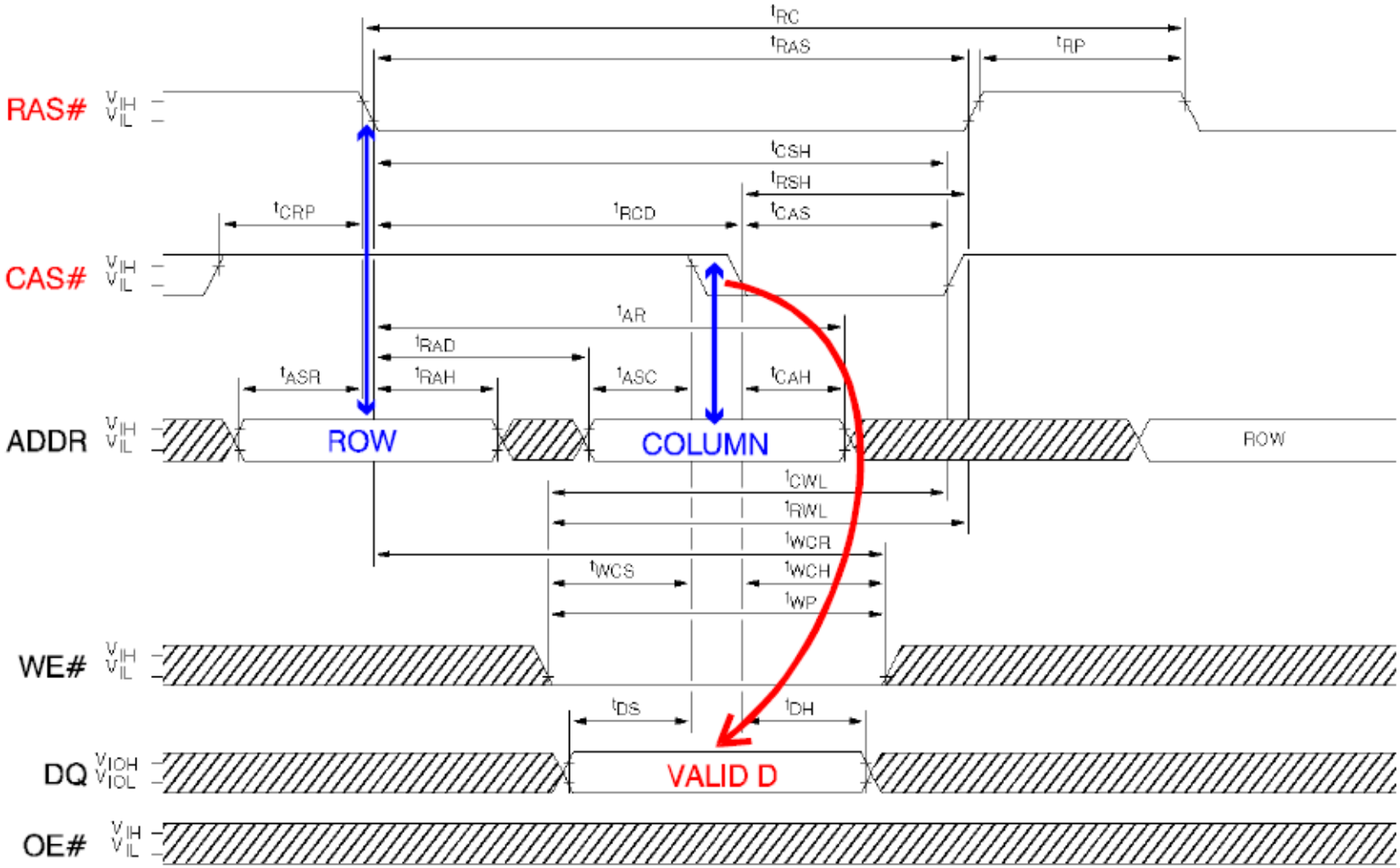
**Dual In-line Memory Module**

- The row address is placed on the address pins via the address bus
- The Row Access Strobe (RAS) is activated, which places the row address onto the Row Address Latch
- The Row Address Decoder selects the proper row to be sent to the sense amps
- The column address is placed on the address pins via the address bus
- The Column Address Strobe (CAS) is activated, which places the column address on the Column Address Latch
- The CAS pin also serves as the Output Enable, so the sense amps place the data from the selected row and column on the Data Out pin

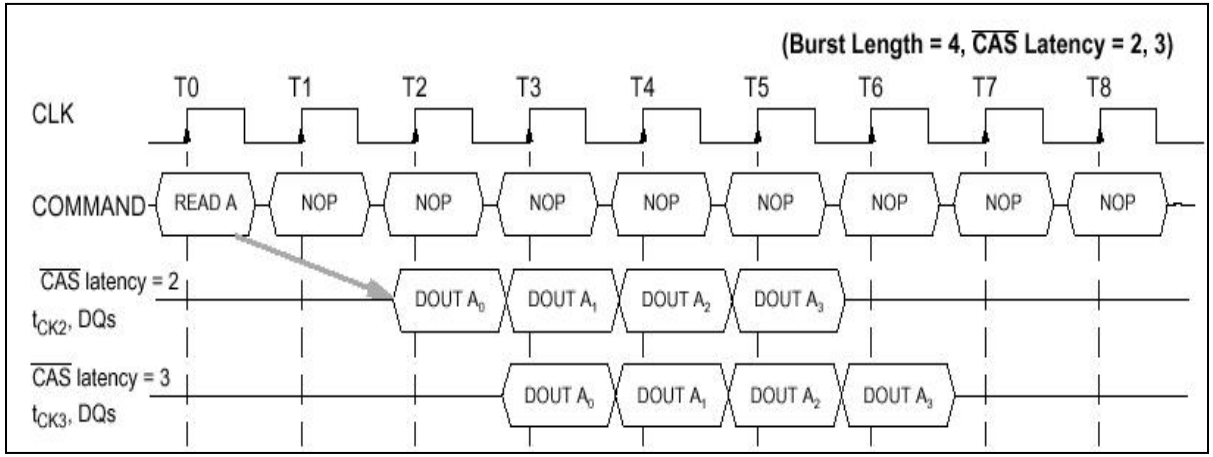
# DRAM read cycle



# DRAM write cycle



# Synchronous DRAM (SDRAMs)



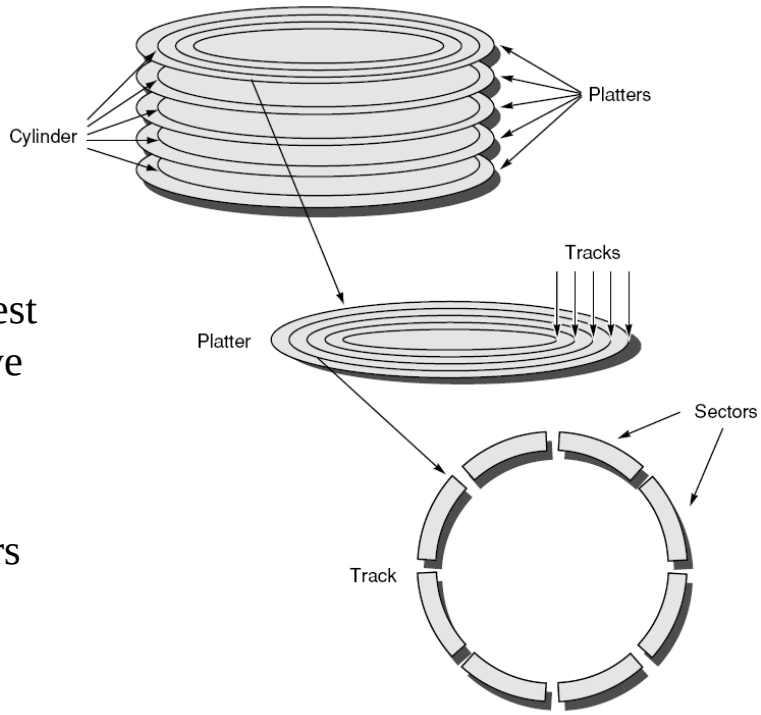

- A specific CLK signal is transmitted over a dedicated control line
- Communication times are based on a multiple of the clock period
- Commands are sent and managed like in a pipeline
- *Burst transfer* (i.e. transfer of chunks of data after a single addressing phase) are possible → High data throughput
- In double data rate (DDR) SDRAM memories, both clock edges are used to transfer data

# Hints on storage memories

# Magnetic disks

- Magnetic disks have dominated nonvolatile storage since 1965. Magnetic disks play two roles in computers:
  - Long-term, nonvolatile storage for files, even when no programs are running
  - A level of the memory hierarchy below main memory used as a backing store for virtual memory during program execution (see later)

- A magnetic "hard "disk consists of a collection of *platters* (generally 1 to 12), rotating at 3,600 to 15,000 revolutions per minute (RPM).
- Platters are metal or glass disks covered with magnetic recording material on both sides.
- Disk diameter is usually 3.5 inches. Traditionally, the biggest disks have the highest performance and the small disks have the lowest price.
- Disk surfaces are divided into 5,000 to 30,000 *tracks*
- Each track in turn is divided into *sectors* (100 to 500 sectors per track).
- A sector is the smallest unit that can be read or written.

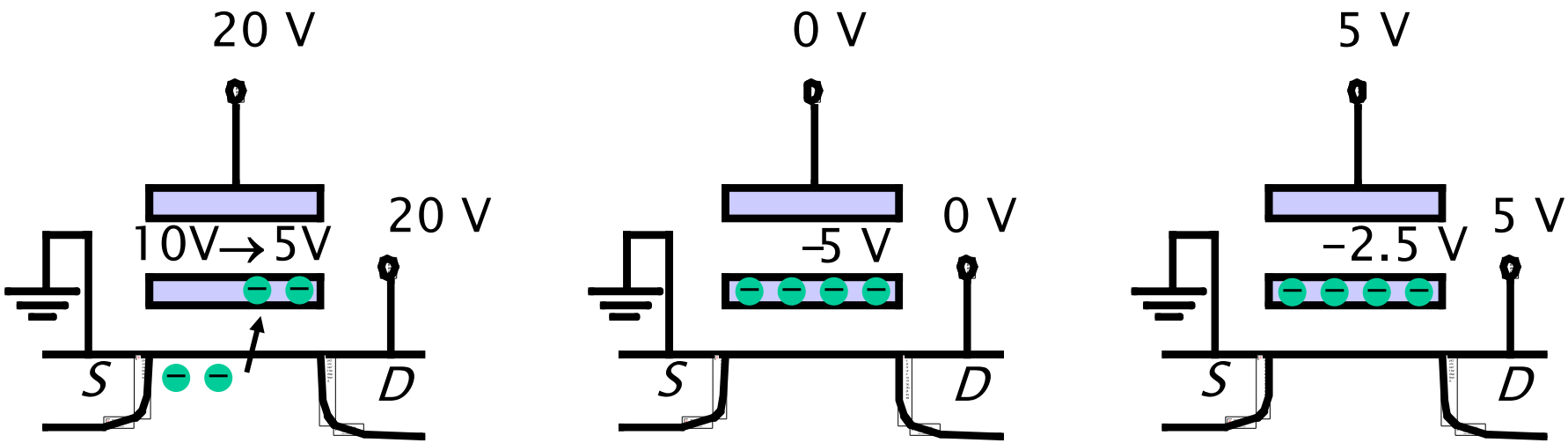


# Magnetic disks performances

- To read and write information into a sector, a movable *arm* containing a *read/write head* is located over each surface.
- *Cylinder:* all the tracks under the arms at a given point on all surfaces.
- *Seek time* ($T_s$): time to move the arm to the desired track (5-12 ms on average)
- *Rotation latency* (or delay) ($T_r$): time for the requested sector to rotate under the head. The average latency to the desired information is obviously halfway around the disk (3-5 ms)
- *Transfer time* ($T_t$)*:* time to transfer a block of bits, typically a sector, under the read-write head. This time is a function of the block size, disk size, rotation speed, recording density of the track, and speed of the electronics. A disk buffer (4-16 MB) can improve $T_t$ by exploiting spatial locality
- *Controller time* ($T_C$)*:* overhead the disk controller imposes in performing an I/O access

$$Access_Time = T_s + T_r + T_t + T_c$$

# Flash ROMs and EEPROMs -1




Avalanche injection

Removing programming voltage leaves charges trapped

Programming results in higher $V_T$

- In Flash ROMs and EEPROMs, a bit cell is made using Floating Gate MOS transistors.
- Memory is written by inducing the tunneling of charge from transistor gate to a floating gate. The floating gate acts as a potential "well" which stores the charge, and the charge cannot move from there without applying an external stimulus (higher voltage).

# Flash ROMs and EEPROMs - 2

- Flash memory is used as a rewritable ROM and main storage in embedded systems, and it is preferred over hard disks due to power and size constraints.

- Compared to disks, Flash memories offer low power consumption (less than 50 mW), can be sold in small sizes, and offer read access times comparable to DRAMs. However unlike DRAMs, writing is much slower and more complicated.

- The primary difference between EEPROM and Flash memory is that Flash restricts write to multi-kilobyte blocks, whereas in EEPROM data are erased/written byte by byte. A block of Flash memory is first electrically erased, and then written with 0s and 1s.

- If the logical data is smaller than the Flash block size, the good data that should survive must be copied to another block before the old block can be erased. Thus, information is organized in Flash as linked lists of blocks.